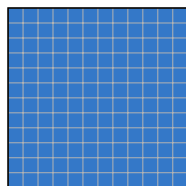


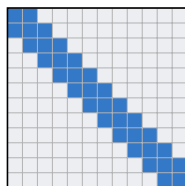
# Handling Long Sequences, Computational Complexity of Transformers, and Fast Attention Mechanisms

Introduction to Large Language Models — Week 11

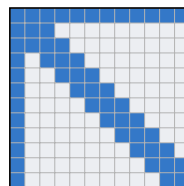
Detailed study notes based on the lecture slides of Prof. Mitesh M. Khapra  
(AI4Bharat, Dept. of CSE, IIT Madras)



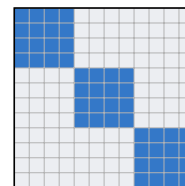
Full  $\mathcal{O}(T^2)$



Local



Global + local



Block

Structure: Motivation → Module 1 (time complexity) → Module 2 (memory) → Module 3  
(fast/approximate attention) → Module 4 (FlashAttention) → Module 5 (fast inference) →  
Practice set

# Contents

|                                                                                 |           |
|---------------------------------------------------------------------------------|-----------|
| <b>How to use these notes</b>                                                   | <b>4</b>  |
| <b>1 Motivation: why long sequences?</b>                                        | <b>5</b>  |
| 1.1 Context length of earlier models . . . . .                                  | 5         |
| 1.2 Sequences in other modalities are long . . . . .                            | 5         |
| 1.3 Text tasks that need long context . . . . .                                 | 5         |
| 1.4 Context lengths of recent models (as listed in the slides) . . . . .        | 5         |
| 1.5 So why is scaling context hard? . . . . .                                   | 6         |
| <b>2 Module 1: Time and space complexity of the attention mechanism</b>         | <b>7</b>  |
| 2.1 Setting up . . . . .                                                        | 7         |
| 2.2 Stage 1: the unnormalised scores $Z = QK^\top$ . . . . .                    | 7         |
| 2.3 Stage 2: the softmax . . . . .                                              | 8         |
| 2.4 Stage 3: multiplying by the values $O = AV$ . . . . .                       | 8         |
| 2.5 Putting it together . . . . .                                               | 8         |
| <b>3 Module 2: Memory requirements</b>                                          | <b>10</b> |
| 3.1 Notation . . . . .                                                          | 10        |
| 3.2 Memory for the $Q, K, V$ matrices . . . . .                                 | 10        |
| 3.3 Memory for the score matrix $Z$ and attention matrix $A$ . . . . .          | 10        |
| 3.4 Key takeaways (Modules 1 and 2) . . . . .                                   | 12        |
| <b>4 Module 3: Fast attention mechanisms</b>                                    | <b>13</b> |
| 4.1 Objective . . . . .                                                         | 13        |
| 4.2 A map of the approaches . . . . .                                           | 13        |
| 4.3 Full attention (the baseline) . . . . .                                     | 13        |
| 4.4 Strided (sliding-window) local attention . . . . .                          | 14        |
| 4.5 Dilated attention . . . . .                                                 | 15        |
| 4.6 Global + local attention (Longformer-style) . . . . .                       | 16        |
| 4.7 Global + random + local attention (BigBird) . . . . .                       | 16        |
| 4.8 Local block attention . . . . .                                             | 17        |
| 4.8.1 Formalising block attention . . . . .                                     | 18        |
| 4.8.2 Computing block attention and its complexity . . . . .                    | 19        |
| 4.8.3 What happens as $n$ grows? . . . . .                                      | 20        |
| 4.8.4 Capturing long-term dependency and performance . . . . .                  | 20        |
| 4.9 Locality Sensitive Hashing attention (Reformer) . . . . .                   | 21        |
| 4.9.1 The idea: attention is approximately a nearest-neighbour search . . . . . | 21        |
| 4.9.2 The catch: shared query–key space . . . . .                               | 22        |

|          |                                                                        |           |
|----------|------------------------------------------------------------------------|-----------|
| 4.9.3    | The hash function . . . . .                                            | 22        |
| 4.9.4    | A 2-D worked example . . . . .                                         | 22        |
| 4.9.5    | The full LSH attention procedure . . . . .                             | 24        |
| 4.9.6    | Complexity . . . . .                                                   | 24        |
| 4.10     | Self-attention is low rank (Linformer) . . . . .                       | 25        |
| 4.10.1   | Where we stand . . . . .                                               | 25        |
| 4.10.2   | The empirical observation . . . . .                                    | 25        |
| 4.10.3   | The Linformer construction . . . . .                                   | 26        |
| 4.11     | Kernel methods (Performer) . . . . .                                   | 28        |
| 4.11.1   | Rewriting softmax attention . . . . .                                  | 28        |
| 4.11.2   | Generalising to a kernel . . . . .                                     | 28        |
| 4.11.3   | How do we compute $\phi(x)$ ? . . . . .                                | 29        |
| 4.11.4   | Vectorised form . . . . .                                              | 30        |
| 4.11.5   | Orthogonal random features . . . . .                                   | 30        |
| 4.11.6   | Complexity . . . . .                                                   | 30        |
| 4.12     | Reduced FLOPs $\neq$ reduced wall-clock time . . . . .                 | 31        |
| <b>5</b> | <b>Module 4: Know your hardware to get wall-clock speed</b>            | <b>32</b> |
| 5.1      | FlashAttention: exact attention with IO-awareness . . . . .            | 32        |
| 5.2      | A toy example: every operation is read, compute, write . . . . .       | 32        |
| 5.3      | Kernel fusion . . . . .                                                | 32        |
| 5.4      | Naive multi-head attention is memory-bound . . . . .                   | 33        |
| 5.5      | The objective, precisely . . . . .                                     | 34        |
| 5.6      | GPU memory hierarchy . . . . .                                         | 34        |
| 5.7      | Idea 1: tiling . . . . .                                               | 34        |
| 5.8      | Safe softmax . . . . .                                                 | 35        |
| 5.9      | Online softmax: a worked example . . . . .                             | 35        |
| 5.10     | The FlashAttention algorithm (forward pass) . . . . .                  | 37        |
| 5.11     | Walk-through: $N = 128$ , $d = 16$ , $B_c = 16$ , $B_r = 32$ . . . . . | 38        |
| 5.12     | HBM access complexity . . . . .                                        | 39        |
| 5.13     | Recomputation (backward pass) . . . . .                                | 39        |
| 5.14     | Did we achieve a wall-clock speed-up? . . . . .                        | 39        |
| 5.15     | Another memory-bound case: inference . . . . .                         | 40        |
| <b>6</b> | <b>Module 5: Fast inference</b>                                        | <b>41</b> |
| 6.1      | Naive autoregressive inference . . . . .                               | 41        |
| 6.2      | KV caching . . . . .                                                   | 41        |
| 6.3      | KV cache size . . . . .                                                | 42        |
| 6.4      | Multi-Query Attention (MQA) . . . . .                                  | 43        |

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| 6.5      | Grouped-Query Attention (GQA) . . . . .        | 43        |
| 6.6      | Paged Attention (vLLM) . . . . .               | 44        |
| <b>7</b> | <b>Consolidated summary and formula sheet</b>  | <b>46</b> |
| 7.1      | Big-picture concept map . . . . .              | 46        |
| 7.2      | Complexity cheat sheet . . . . .               | 46        |
| 7.3      | Formula sheet . . . . .                        | 47        |
| 7.4      | The ten sentences to remember . . . . .        | 47        |
| <b>8</b> | <b>Practice set</b>                            | <b>48</b> |
| 8.1      | Part A: Numerical problems . . . . .           | 48        |
| 8.2      | Part A: Solutions . . . . .                    | 49        |
| 8.3      | Part B: Subjective questions . . . . .         | 52        |
| 8.4      | Part B: Model answers . . . . .                | 52        |
| 8.5      | Part C: Multiple-choice questions . . . . .    | 55        |
| 8.6      | Part C: Answer key with explanations . . . . . | 57        |

## How to use these notes

Each module follows the same pattern: the *problem* it attacks, the *intuition*, the *mathematics* (with every matrix shape written underneath it), a *diagram*, and a short summary. Coloured boxes mean:

| Box                            | Purpose                                                          |
|--------------------------------|------------------------------------------------------------------|
| <b>Intuition</b> (blue)        | the mental picture behind an idea                                |
| <b>Key idea</b> (green)        | the single sentence to remember                                  |
| <b>Derivation</b> (grey)       | step-by-step maths                                               |
| <b>Note</b> (orange)           | extra context beyond the slides, or clarifications of the slides |
| <b>Common pitfall</b> (red)    | mistakes students typically make in exams                        |
| <b>Worked example</b> (purple) | a small numeric walk-through                                     |

Shapes are written as  $Q_{T \times d}$ , i.e. the shape is printed below the symbol. Colours in diagrams are consistent throughout: **queries**, **keys**, **values**, **attention/scores**.

## Master notation table

| Symbol                               | Meaning                                                                                                                            |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| $T$ (also $N, L$ )                   | context / sequence length (number of tokens). FlashAttention papers use $N$ , Performer figures use $L$ ; all mean the same thing. |
| $d = d_q = d_k$                      | dimension of a (projected) query/key vector of one head                                                                            |
| $d_v$                                | dimension of a value vector (usually $d_v = d$ )                                                                                   |
| $n_h$                                | number of attention heads                                                                                                          |
| $d_{\text{model}}$                   | model (embedding) dimension, $d_{\text{model}} = n_h d$                                                                            |
| $d_{ff}$                             | hidden size of the feed-forward layer, typically $4d_{\text{model}}$ (e.g. 2048 when $d_{\text{model}} = 512$ )                    |
| $B$                                  | batch size                                                                                                                         |
| $N_p$                                | number of parameters (the slides write $N$ ; renamed here to avoid a clash with sequence length)                                   |
| $ V $                                | vocabulary size                                                                                                                    |
| $Q, K, V$                            | query, key, value matrices, each $T \times d$ per head                                                                             |
| $Z = S = QK^\top$                    | unnormalised scores, $T \times T$                                                                                                  |
| $A = P = \text{softmax}(Z/\sqrt{d})$ | attention (probability) matrix, $T \times T$ , rows sum to 1                                                                       |
| $O = AV$                             | output of attention, $T \times d$                                                                                                  |
| $c$                                  | window size / number of keys attended per query in sparse attention                                                                |
| $n$                                  | number of blocks in block attention                                                                                                |
| $k$                                  | projected length in Linformer                                                                                                      |
| $r$                                  | number of random features in Performer                                                                                             |
| $M$                                  | SRAM size in FlashAttention (also used for a mask matrix in block attention — context disambiguates)                               |
| $B_r, B_c$                           | row (query) and column (key/value) tile sizes in FlashAttention                                                                    |

# 1 Motivation: why long sequences?

## 1.1 Context length of earlier models

The models studied so far (BERT, GPT, GPT-2, T5, ...) have a small fixed context window: **512** tokens (BERT, GPT-1) or **1024** tokens (GPT-2). Anything outside the window is simply invisible to the model.

## 1.2 Sequences in other modalities are long

Transformers are not only for text. Anything that can be laid out as a sequence can be fed to them:

- **Audio.** A 10-second clip sampled at 8000 Hz has  $10 \times 8000 = 80,000$  samples, so a sequence of length 80,000.
- **Images (pixels as a sequence).** A  $100 \times 100$  colour image has 3 channels:  $100 \times 100 \times 3 = 30,000$  values, so a sequence of length 30,000.

## 1.3 Text tasks that need long context

- **Summarising a book** of about 64,000 words. With roughly 1.3 tokens per word this is more than 80K tokens.
- **Generating a story** of at least 5000 words: the model must stay consistent (names, plot) across thousands of tokens.
- **Question answering over a 1000-page manual:** the answer may be on page 734, and the model must be able to “see” it.

### Intuition

A context window works like short-term memory. With a 512-token window, reading a book is like reading one page, forgetting it, then reading the next. To reason across a whole document, the model has to hold all of it at once, which means *attending over long sequences*.

## 1.4 Context lengths of recent models (as listed in the slides)

| Model                       | Context length                  |
|-----------------------------|---------------------------------|
| LLaMA-2                     | 4K                              |
| Mistral-7B                  | 8K                              |
| GPT-4, Gemini 1.0           | 32K                             |
| MosaicML MPT                | 65K                             |
| Anthropic Claude            | 100K                            |
| Gemini 1.5 Pro, GPT-4 Turbo | 128K                            |
| Claude 2.1                  | 200K                            |
| Gemini 1.5 Pro (limited)    | 2M (public), 10M (experimental) |

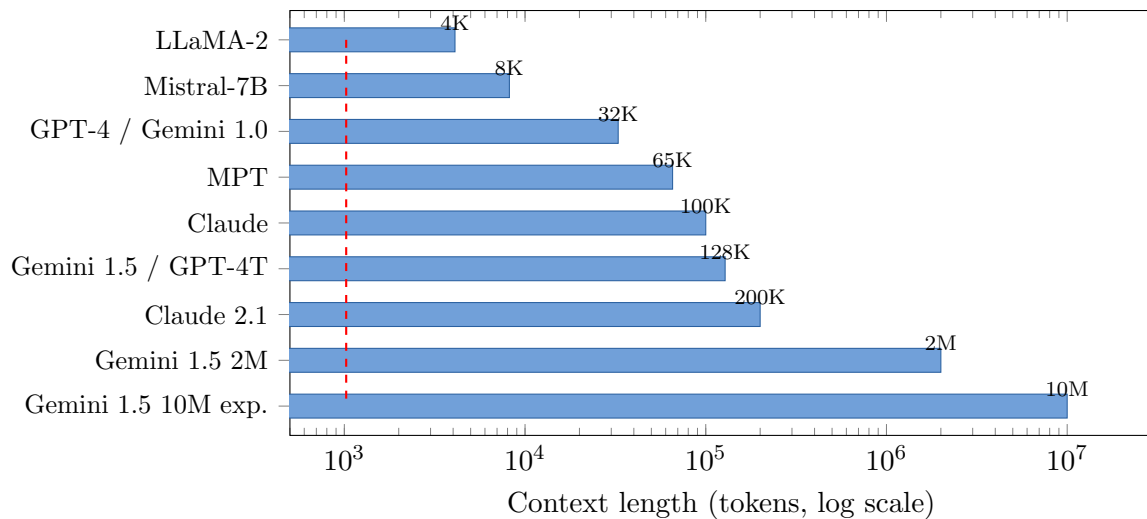


Figure 1: Context lengths from the slides. The dashed red line marks GPT-2's 1024 tokens. The spread from 1K to 10M covers four orders of magnitude.

### Note

This table is a snapshot from when the lecture was made. Newer models have pushed context further (many frontier models now offer windows from several hundred thousand to over a million tokens). The *techniques* in this week (sparse attention, FlashAttention, KV-cache optimisations such as GQA and paged attention) are exactly the tools that made these jumps possible.

## 1.5 So why is scaling context hard?

One of the main obstacles is the **computational complexity of self-attention**. Every token attends to every other token, so the work grows with the *square* of the sequence length. Before trying to fix that, we measure it precisely:

- **Module 1:** time (FLOP) complexity of attention.
- **Module 2:** memory needed to train Transformers.
- **Module 3:** algorithms that approximate attention in sub-quadratic time.
- **Module 4:** FlashAttention, which is *exact* attention made fast by respecting GPU hardware.
- **Module 5:** making *inference* fast (KV cache, MQA, GQA, paged attention).

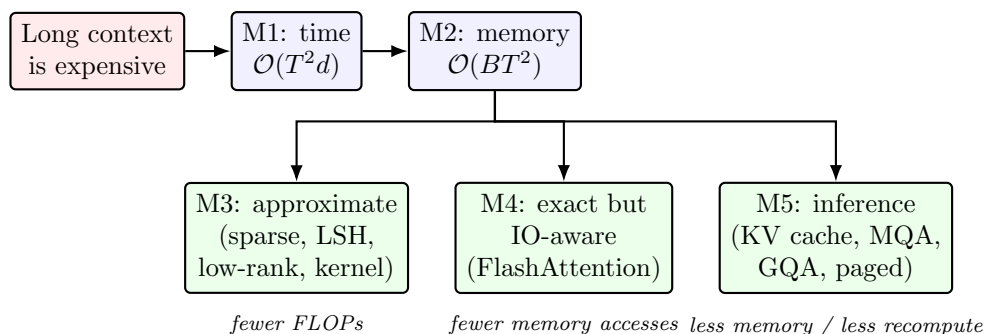


Figure 2: How the modules of this week fit together.

## 2 Module 1: Time and space complexity of the attention mechanism

### 2.1 Setting up

Consider a single attention head. The input sequence of  $T$  tokens has already been projected into queries, keys and values:

$$\begin{matrix} Q \\ T \times d \end{matrix} = X W_Q, \quad \begin{matrix} K \\ T \times d \end{matrix} = X W_K, \quad \begin{matrix} V \\ T \times d \end{matrix} = X W_V, \quad \begin{matrix} X \\ T \times d_{\text{model}} \end{matrix}, \quad \begin{matrix} W_Q \\ d_{\text{model}} \times d \end{matrix}.$$

- Context window size:  $T$ .
- Size of the projected embeddings:  $d = d_q = d_k$  (query and key must share a dimension so we can take dot products).

Scaled dot-product attention is

$$\begin{matrix} O \\ T \times d \end{matrix} = \underbrace{\text{softmax}\left(\frac{Q K^\top}{\sqrt{d}}\right)}_{\begin{matrix} A \\ T \times T \end{matrix}} \begin{matrix} V \\ T \times d \end{matrix}$$

We count the operations in three stages: (1)  $Z = QK^\top$ , (2) the softmax, and (3)  $AV$ .

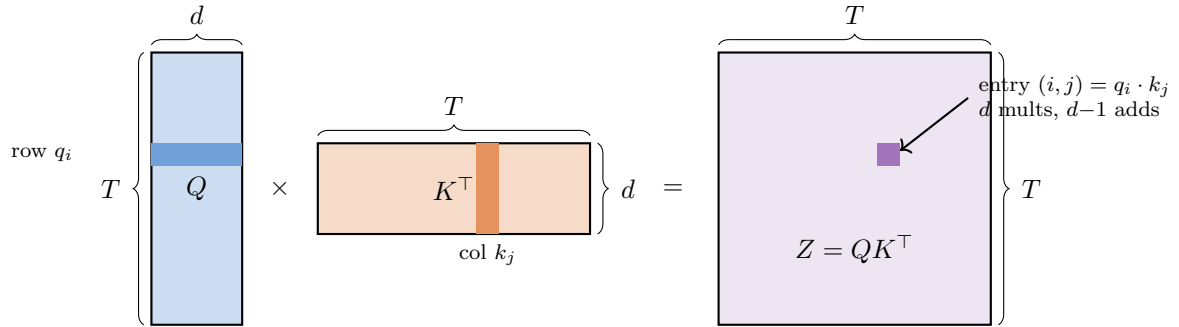


Figure 3: Computing  $Z = QK^\top$ . Each of the  $T^2$  entries is a dot product of a  $d$ -dimensional query with a  $d$ -dimensional key.

### 2.2 Stage 1: the unnormalised scores $Z = QK^\top$

**Derivation: cost of  $QK^\top$**

**One entry.**  $Z_{ij} = q_i \cdot k_j = \sum_{t=1}^d q_{it} k_{jt}$  needs

$d$  multiplications and  $d - 1$  additions.

**One row.** Row  $i$  of  $Z$  is  $q_i$  dotted with all  $T$  keys, so row  $i \in \mathbb{R}^{1 \times T}$  costs

$T \cdot d$  multiplications and  $T \cdot (d - 1)$  additions.

**Whole matrix.** There are  $T$  rows:

$$T^2 d \text{ multiplications} + T^2 (d - 1) \text{ additions} \approx 2T^2 d \text{ FLOPs} = \boxed{\mathcal{O}(T^2 d)}.$$

### Note: why $T^2$ dominates

In practice the number of tokens is larger than the head dimension,  $T > d$  (e.g.  $d = 64$  or  $128$ ,  $T = 2048$  to  $10^6$ ). So  $T^2 d$  grows much faster with  $T$  than with  $d$ . Doubling  $d$  doubles the cost; doubling  $T$  *quadruples* it.

## 2.3 Stage 2: the softmax

$$A_{T \times T} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right), \quad A_{ij} = \frac{\exp(Z_{ij}/\sqrt{d})}{\sum_{j'=1}^T \exp(Z_{ij'}/\sqrt{d})}.$$

### Derivation: cost of the row-wise softmax

For one row of  $Z$  (length  $T$ ): scale by  $1/\sqrt{d}$  ( $T$  ops), exponentiate ( $T$  ops), sum ( $T - 1$  ops), divide ( $T$  ops), about  $4T$  ops in total, i.e.  $\mathcal{O}(T)$  per row.

There are  $T$  rows, so softmax costs  $T \cdot \mathcal{O}(T) = \mathcal{O}(T^2)$ .

Hence computing  $A$  costs  $\mathcal{O}(T^2 d) + \mathcal{O}(T^2) = \boxed{\mathcal{O}(T^2 d)}$ .

The softmax has fewer *FLOPs* than the matmul by a factor of  $d$ . As Module 4 shows, it can still take *longer* in wall-clock time, because it is memory-bound.

## 2.4 Stage 3: multiplying by the values $O = AV$

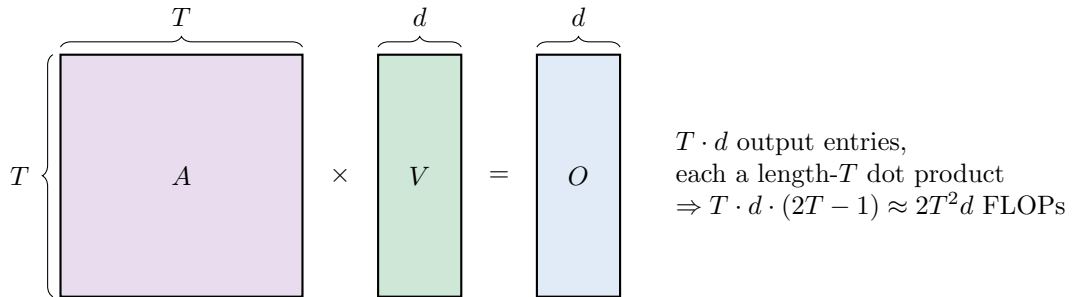


Figure 4: The final matmul  $O = AV$  is also quadratic in  $T$ .

### Derivation: cost of $AV$

$O_{it} = \sum_{j=1}^T A_{ij} V_{jt}$  takes  $T$  multiplications and  $T - 1$  additions. There are  $T \times d$  output entries, so the total is  $Td(2T - 1) = \boxed{\mathcal{O}(T^2 d)}$ .

## 2.5 Putting it together

$$\underbrace{\mathcal{O}(T^2 d)}_{QK^\top} + \underbrace{\mathcal{O}(T^2)}_{\text{softmax}} + \underbrace{\mathcal{O}(T^2 d)}_{AV} = \boxed{\mathcal{O}(T^2 d)}.$$

### Key idea

The entire self-attention block is **quadratic in the sequence length  $T$**  (and linear in  $d$ ). The  $T \times T$  matrix is the culprit: every token interacts with every other token.

**Note: beyond the slides — the full layer**

A Transformer layer also has other costs. With  $d_{\text{model}} = n_h d$ :

- QKV and output projections:  $4 \cdot T d_{\text{model}}^2$  multiply-adds  $\Rightarrow \mathcal{O}(T d_{\text{model}}^2)$ , which is *linear* in  $T$ .
- Attention over all heads:  $n_h \cdot \mathcal{O}(T^2 d) = \mathcal{O}(T^2 d_{\text{model}})$ , which is *quadratic* in  $T$ .
- Feed-forward:  $2 \cdot T d_{\text{model}} d_{\text{ff}} = 8 T d_{\text{model}}^2 \Rightarrow \mathcal{O}(T d_{\text{model}}^2)$ , linear in  $T$ .

Attention dominates once  $T^2 d_{\text{model}} \gtrsim T d_{\text{model}}^2$ , i.e. roughly when  $T \gtrsim d_{\text{model}}$  (up to constant factors). For GPT-2 small ( $d_{\text{model}} = 768$ ,  $T = 1024$ ) the two are comparable. For  $T = 100\text{K}$  attention dwarfs everything else. That is why long-context work focuses on the attention block.

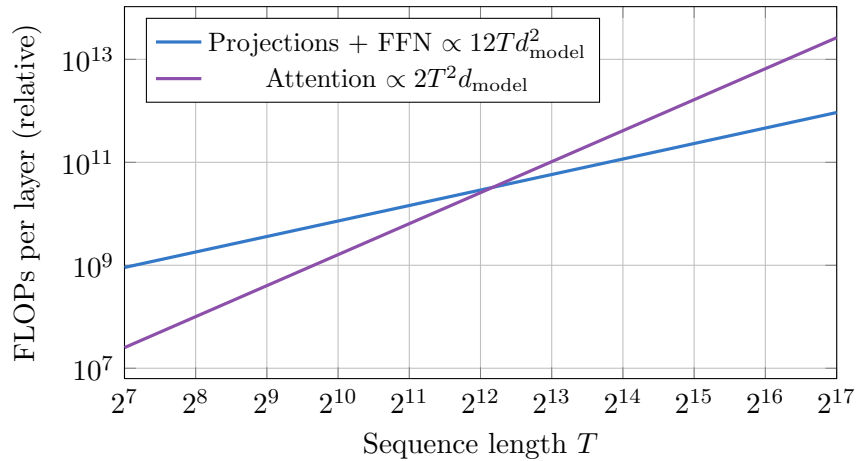


Figure 5: Illustrative FLOP scaling for  $d_{\text{model}} = 768$  (log-log axes). The linear parts have slope 1, attention has slope 2, and the lines cross at  $T = 6d_{\text{model}} \approx 4.6\text{K}$ .

**Summary: Module 1**

- One dot product:  $d$  mults,  $d - 1$  adds. One row of  $Z$ :  $Td$  mults,  $T(d - 1)$  adds.
- $QK^\top$ :  $\mathcal{O}(T^2 d)$ . Softmax:  $\mathcal{O}(T)$  per row,  $\mathcal{O}(T^2)$  total.  $AV$ :  $\mathcal{O}(T^2 d)$ .
- Self-attention:  $\mathcal{O}(T^2 d)$  time, quadratic in context length.

### 3 Module 2: Memory requirements

#### 3.1 Notation

|                      |                                                                                          |
|----------------------|------------------------------------------------------------------------------------------|
| $B$                  | batch size                                                                               |
| $T$                  | context (sequence) length                                                                |
| $n_h$                | number of attention heads                                                                |
| $d_{\text{model}}$   | dimension of the input embedding (e.g. 512)                                              |
| $d = d_{\text{att}}$ | per-head dimension, $d = d_{\text{model}}/n_h$                                           |
| $d_{\text{ff}}$      | feed-forward hidden dimension, typically $d_{\text{ff}} = 4d_{\text{model}}$ (e.g. 2048) |
| $N_p$                | number of parameters in the model                                                        |
| $ V $                | number of tokens in the vocabulary                                                       |

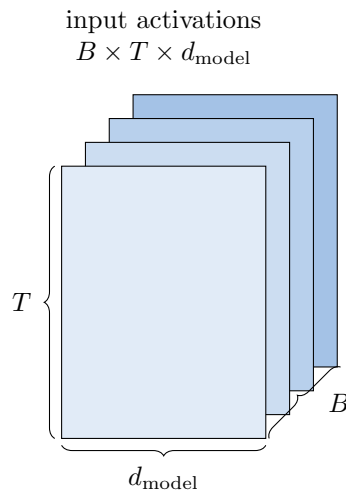


Figure 6: A batch of activations is a  $B \times T \times d_{\text{model}}$  tensor.

#### 3.2 Memory for the $Q, K, V$ matrices

Each head has its own  $Q, K, V$ , each of shape  $T \times d$ :

$$\text{mha} = \begin{pmatrix} Q \\ T \times d \end{pmatrix} + \begin{pmatrix} K^\top \\ d \times T \end{pmatrix} + \begin{pmatrix} V \\ T \times d \end{pmatrix} n_h.$$

##### Derivation: $Q, K, V$ memory

$$\begin{aligned} \text{mha} &= 3 n_h (T \times d) && \text{(three matrices, } Td \text{ numbers each, for } n_h \text{ heads)} \\ &= 3T \times (n_h d) \\ &= 3T \times d_{\text{model}} && \because n_h d = d_{\text{model}}. \end{aligned}$$

For a batch of  $B$  samples:  $\boxed{3BTd_{\text{model}}}$  numbers. This is *linear* in  $T$ .

#### 3.3 Memory for the score matrix $Z$ and attention matrix $A$

$$Z = QK^\top \in \mathbb{R}^{T \times T}, \quad A = \text{softmax}(QK^\top) \in \mathbb{R}^{T \times T}.$$

### Derivation: $Z$ and $A$ memory

One head stores a  $T \times T$  matrix, so  $n_h$  heads store  $n_h T^2$ . For a batch:  $B n_h T^2$  for  $Z$ , and another  $B n_h T^2$  for  $A$  (both are usually kept for the backward pass).

$$\text{mem}(Z) = \text{mem}(A) = \boxed{B n_h T^2}.$$

### Key idea

Memory scales **linearly with the batch size  $B$**  and **quadratically with the context length  $T$** .

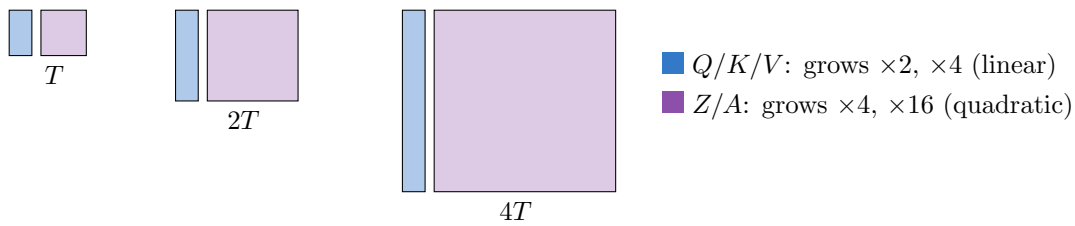


Figure 7: Doubling  $T$  doubles the  $QKV$  memory but quadruples the attention-matrix memory.

### Worked example: how bad is $T^2$ ?

Take  $B = 8$ ,  $n_h = 12$ , fp32 (4 bytes).

- $T = 1024$ :  $\text{mem}(A) = 8 \cdot 12 \cdot 1024^2 = 100.7\text{M numbers} \times 4 \text{ B} \approx 403 \text{ MB per layer}$ . With  $Z$  as well, about 0.8 GB per layer.
- $T = 8192$  ( $8\times$  longer):  $64\times$  more,  $\approx 25.8 \text{ GB per layer}$  for  $A$  alone. A 12-layer model would need about 619 GB just for  $Z$  and  $A$ , which no single GPU has.

By contrast  $QKV$  at  $T = 8192$ ,  $d_{\text{model}} = 768$  is  $3 \cdot 8 \cdot 8192 \cdot 768 \cdot 4 \text{ B} \approx 0.6 \text{ GB per layer}$ .

### Note: beyond the slides — the rest of training memory

Training memory has four parts:

1. **Parameters:**  $N_p$  numbers. For a decoder,  $N_p \approx 12 L d_{\text{model}}^2 + |V| d_{\text{model}}$  ( $L$  layers;  $4d_{\text{model}}^2$  for attention projections plus  $8d_{\text{model}}^2$  for the FFN with  $d_{\text{ff}} = 4d_{\text{model}}$ ; plus embeddings).
2. **Gradients:** another  $N_p$ .
3. **Optimizer state:** Adam keeps two moments, so  $2N_p$ . In fp32 that gives about 16 bytes/parameter for weights + grads + Adam.
4. **Activations:** everything saved for the backward pass:  $\mathcal{O}(BTd_{\text{model}})$  for most tensors plus  $\mathcal{O}(Bn_h T^2)$  for the attention matrices (and dropout masks on them).

Items 1–3 do not depend on  $T$ . Item 4 does, and the  $T^2$  part is what blows up for long contexts. FlashAttention (Module 4) removes the need to store the  $T^2$  matrices at all.

### 3.4 Key takeaways (Modules 1 and 2)

#### Summary

- The complexity of self-attention is **quadratic** in the length of the input sequence:  $\mathcal{O}(T^2d)$ .
- Memory scales up **linearly** with batch size  $B$  and **quadratically** with context length  $T$ :  $\mathcal{O}(Bn_hT^2)$  for the attention matrices.
- The next question: *how do we reduce the time complexity so models can handle long sequences during both training and inference?*

## 4 Module 3: Fast attention mechanisms

### 4.1 Objective

- **What we have:** full attention with complexity  $\mathcal{O}(T^2d)$ .
- **What we want:** sub-quadratic complexity  $\mathcal{O}(cTd)$  with  $c \ll T$ .

The natural move is to **localise** attention: each query attends to a subset of keys instead of all  $T$ . There are many ways to choose the subset, which gives a family of approaches that *approximate* full attention.

#### Intuition: why localising is not crazy

A study of BERT on NLP tasks (Clark et al., “What does BERT look at?”) found that

1. **neighbouring inner products are extremely important:** many heads mostly attend to the previous/next token or a small neighbourhood;
2. **not all heads attend to all tokens:** attention maps are largely sparse, with most of the probability mass on a few keys.

If most of the  $T^2$  entries of  $A$  are nearly zero anyway, computing them is wasted work. Skip them.

### 4.2 A map of the approaches

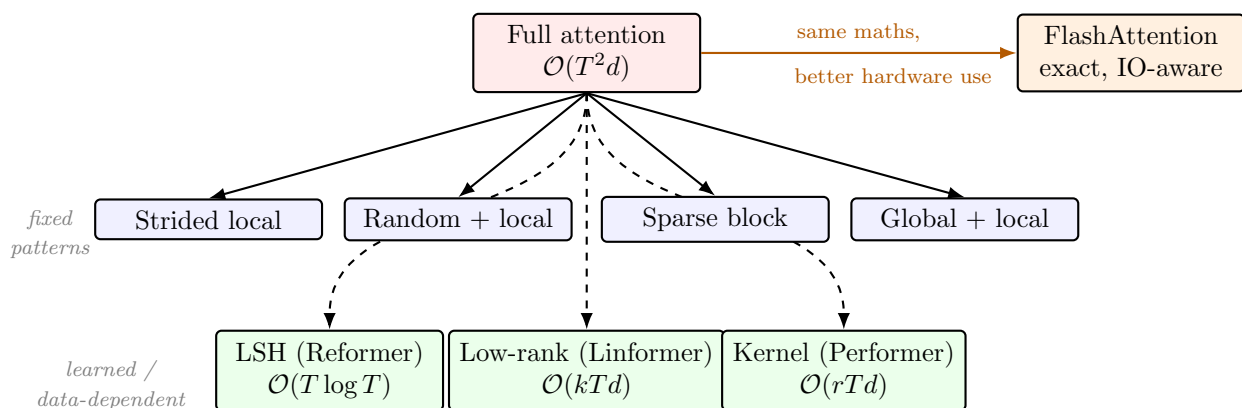


Figure 8: Approximations to full attention (blue: fixed sparsity patterns, green: data-dependent or low-rank approximations) and FlashAttention, which keeps attention exact and speeds it up through hardware awareness.

### 4.3 Full attention (the baseline)

Running example: the sentence “Our planet is a lonely speck” ( $T = 6$ ).

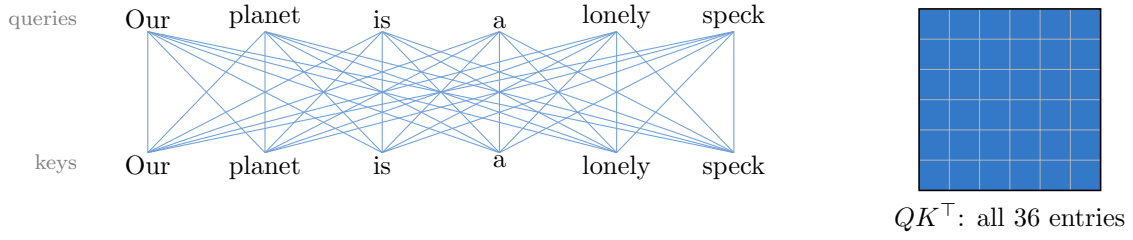


Figure 9: Full attention: every word attends to every other word in the sequence (left). The corresponding  $QK^\top$  mask is dense (right).

- Computational complexity:  $\mathcal{O}(T^2d)$ . Memory complexity:  $\mathcal{O}(T^2)$ .
- Our focus here is on reducing the **computational** complexity.
- One way is to make it **sparse**: attend to a subset of words instead of all of them. Each query always keeps the score for itself ( $q_i^\top k_i$ ). If each query attends to  $c < T$  keys, the cost drops to

$$\mathcal{O}(c \times T \times d), \quad c < T.$$

#### 4.4 Strided (sliding-window) local attention

Every word attends to a window of  $c$  words:  $\lfloor c/2 \rfloor$  to the left,  $\lfloor c/2 \rfloor$  to the right, and itself.

$$M_{ij} = \begin{cases} 1 & |i - j| \leq \lfloor c/2 \rfloor \\ 0 & \text{otherwise} \end{cases} \implies \text{cost } \mathcal{O}(cTd) \text{ (linear in } T \text{ for fixed } c).$$

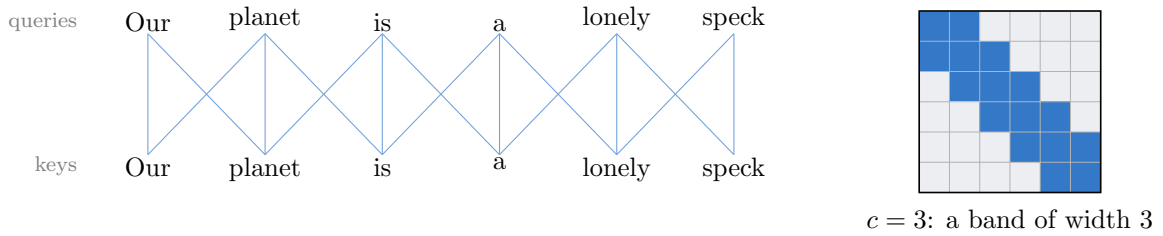


Figure 10: Strided local attention with  $c = 3$ : each word sees only its immediate neighbours. The mask is a diagonal band,  $\mathcal{O}(3Td)$ .

**Varying the window per layer.** We can use different window sizes in different layers, e.g.  $c = 3$  in lower layers ( $\mathcal{O}(3Td)$ ),  $c = 5$  higher up ( $\mathcal{O}(5Td)$ ), and *full (global) attention in the top-most layer*.

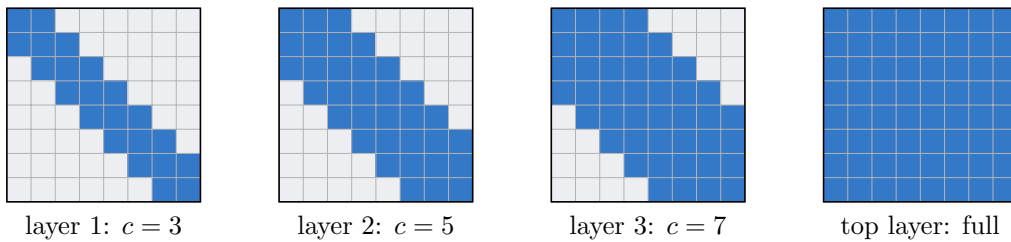
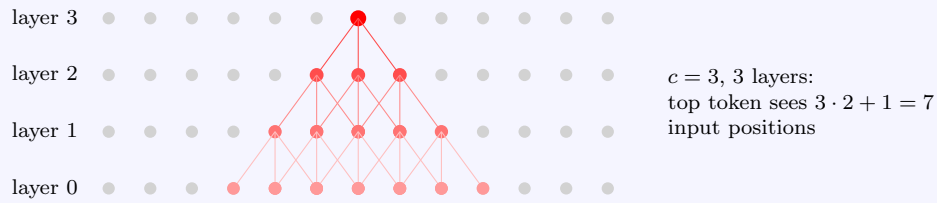


Figure 11: Growing windows across layers. Only the last layer pays the  $\mathcal{O}(T^2)$  price.

### Intuition: receptive field, just like CNNs

Local attention works like a 1-D convolution. One layer with window  $c$  lets a token see  $c$  positions. Stacking layers widens the view: after  $L$  layers of window  $c$ , information can flow across

$$\text{receptive field} = L(c - 1) + 1 \text{ tokens.}$$



This is why local attention can still model long-range structure given enough depth, and why putting one global layer at the top is so effective.

### 4.5 Dilated attention

Like *dilated convolutions*: the window skips positions. With dilation rate  $\delta$ , token  $i$  attends to  $i$ ,  $i \pm \delta$ ,  $i \pm 2\delta, \dots$  (still  $c$  keys in total). Different attention heads can use different dilation rates, so some heads look at neighbours and others look further away.

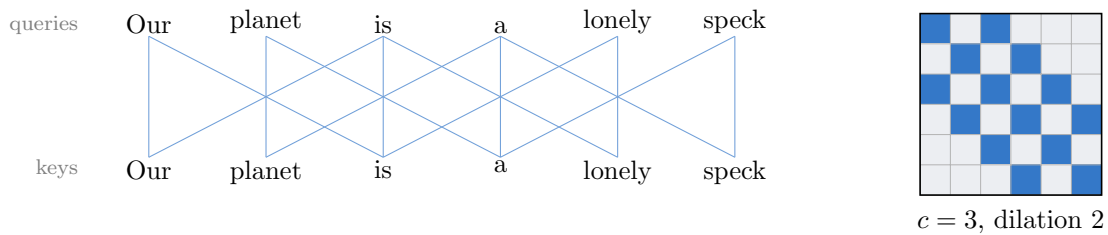


Figure 12: Dilated attention ( $c = 3$ , dilation 2): each word attends to itself and the words two positions away. The window covers a wider span with the same number of keys.

#### Note

The computational complexity does **not** change: still  $\mathcal{O}(3Td)$  for  $c = 3$ . Only the *span* of each window grows.

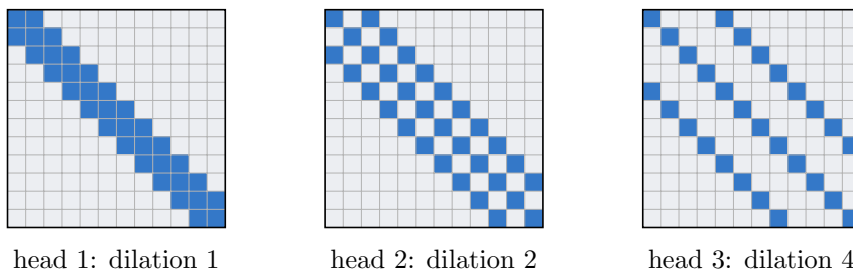


Figure 13: Different heads using different dilation rates. Each head costs the same, but together they cover several ranges.

## 4.6 Global + local attention (Longformer-style)

Some tokens need to see the whole sequence. The `[cls]` token, whose final state is used for classification, needs full attention to perform well on downstream tasks. So:

- **A few words are global:** they attend to every token, *and* every token attends to them (symmetric).
- **Every other word** uses a local window of  $c$  words ( $\lfloor c/2 \rfloor$  left,  $\lfloor c/2 \rfloor$  right).
- The global tokens are **set manually, depending on the task**: `[cls]` for classification, the question tokens for QA, and so on.

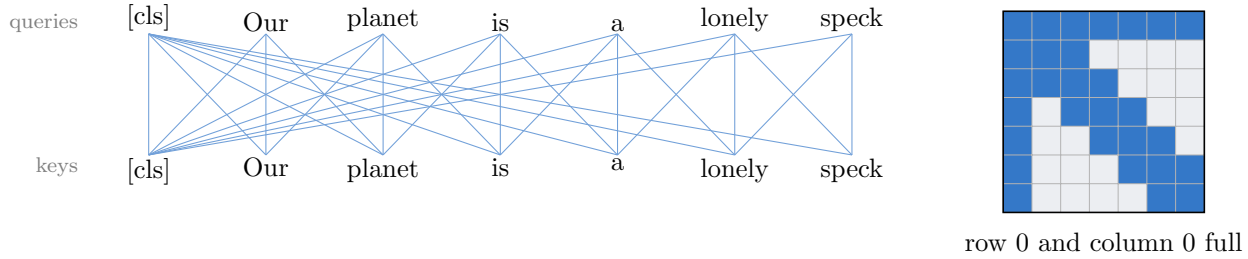


Figure 14: Global + local attention with `[cls]` as the global token. The mask is a band plus one full row and one full column. Cost:  $\mathcal{O}((3T + T)d)$ .

### Derivation: cost of global + local

Local band with  $c = 3$ : about  $3T$  entries. The global token's row adds  $T$  entries, and its column adds another  $T$  (minus overlaps). The slide counts this as  $\mathcal{O}((3T + T)d)$ . More generally, with  $g$  global tokens:

$$\text{cost} \approx (cT + 2gT) d = \mathcal{O}((c + 2g) T d),$$

which is still **linear** in  $T$  as long as  $g$  and  $c$  are constants.

## 4.7 Global + random + local attention (BigBird)

The **BigBird** model adds a third component: each query also attends to a few **randomly chosen** tokens.

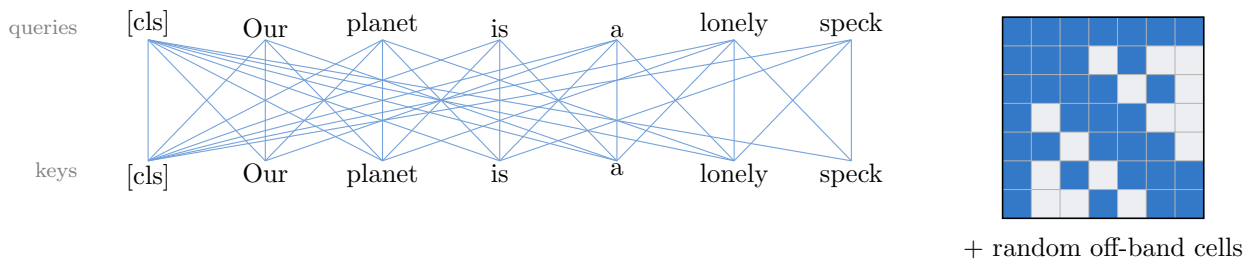


Figure 15: BigBird: global + local + random. Cost  $\mathcal{O}((3T + T + T)d)$ , still linear in  $T$ .

### Intuition: why random edges help

View attention as a *graph* whose nodes are tokens and whose edges are allowed attention links. A local band graph has large diameter: information needs about  $T/c$  layers to cross the sequence. Adding a few random edges turns it into a small-world, expander-like graph in which any two nodes are connected by a short path ( $\mathcal{O}(\log T)$  hops). Global tokens act as hubs,

so every pair of tokens is at most 2 hops apart through [cls]. BigBird's authors show that this combination keeps the theoretical expressiveness of full attention (universal approximation, Turing completeness) at linear cost.

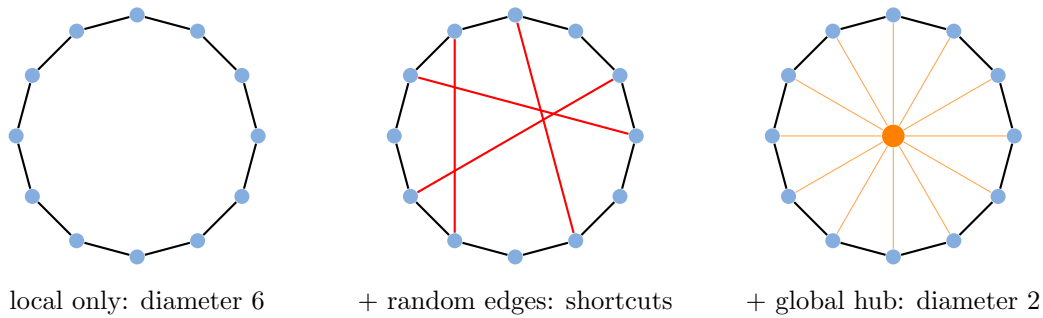


Figure 16: Attention viewed as a graph (sequence drawn as a ring for clarity).

## 4.8 Local block attention

Another way to sparsify is to **divide the sequence into  $n$  blocks** and compute attention *independently inside each block* (as in *Generating Wikipedia by Summarizing Long Sequences* and BlockBERT).

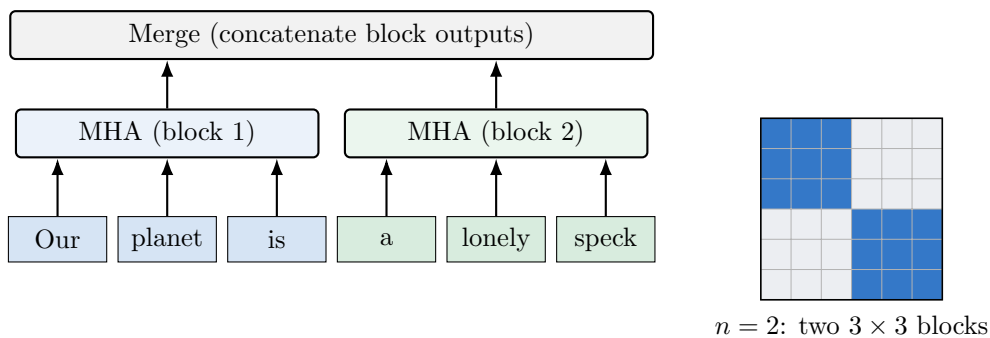


Figure 17: Local block attention with  $n = 2$  blocks: each half of the sentence attends only within itself. The mask is block-diagonal.

### Derivation: complexity of local block attention

Each block has  $T/n$  tokens, so its score matrix is  $(T/n) \times (T/n)$ . With  $n$  blocks:

$$\# \text{entries} = n \cdot \left(\frac{T}{n}\right)^2 = \frac{T^2}{n} \implies \mathcal{O}\left(\frac{T^2}{n} d\right).$$

For  $n = 2$  this gives  $\mathcal{O}(T^2/2)$  entries: half the work of full attention.

Typical values are small, e.g.  $n = 2, 3, 4$ . For a context length of 512 the slides quote block sizes of 256, 128, 64, which are reasonable context sizes for the majority of NLP tasks.

### Note: a small inconsistency in the slides

Strictly,  $512/n$  for  $n = 2, 3, 4$  gives 256,  $\approx 171$ , 128. The sizes 256, 128, 64 correspond to  $n = 2, 4, 8$ . The point stands either way: each block still has a few hundred tokens of context.

**Different patterns per head/layer.** Blocks do not have to be paired diagonally ( $Q$ -block  $i$  with  $K$ -block  $i$ ). Query block  $i$  can be paired with key block  $\pi(i)$  for some permutation  $\pi$ , and each head/layer can use a *different* permutation. Across heads this captures **long-range** dependencies while each head stays cheap.

**Note: how many permutations?**

The slide says that for  $n = 4$  blocks “we can permute these blocks in 16 possible ways”. The number of permutations of  $n$  objects is  $n!$ , so  $n = 4$  gives  $4! = 24$  possible pairings. The number 16 is  $n^2$ , the number of blocks in the  $4 \times 4$  block grid of  $QK^\top$ . In an exam, use  $n!$  for the number of permutations and  $n^2$  for the number of blocks.

### 4.8.1 Formalising block attention

**Recall full attention.**  $Q$  and  $K^\top$  give  $QK^\top$ .  
 $T \times d_k$   $d_k \times T$   $T \times T$

Divide  $Q$  into  $n$  row-blocks  $Q_0, \dots, Q_{n-1}$  and  $K$  into  $n$  row-blocks  $K_0, \dots, K_{n-1}$ . Then  $QK^\top$  splits into  $n^2$  blocks, and block  $(i, j)$  is  $Q_i K_j^\top$ :

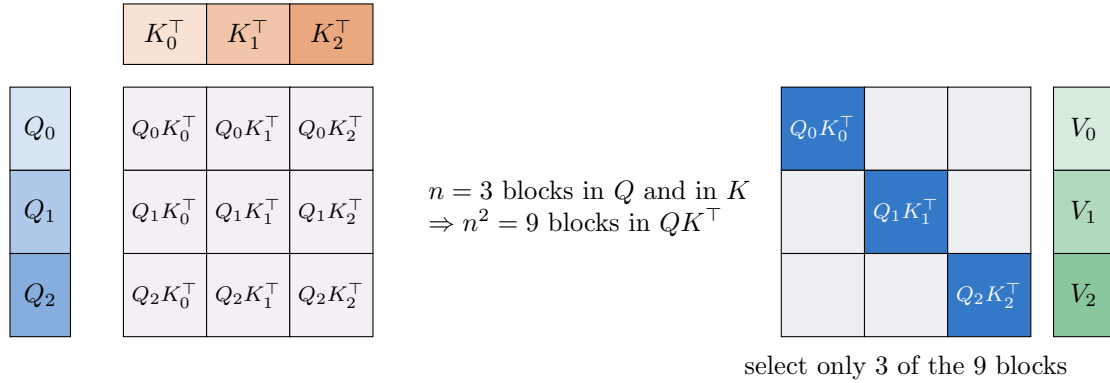


Figure 18: Left: the  $n^2$  blocks of  $QK^\top$ . Right: block attention computes only  $n$  of them (here the diagonal,  $\pi = \text{identity}$ ), each paired with the matching value block.

Suppose we want to select only 3 of the  $n^2 = 9$  blocks, one per query block. There are many ways to do this. We formalise the choice with a **permutation**.

**The permutation.** Let  $\pi = \text{perm}(0, 1, 2, \dots, n-1)$  and write  $\pi(k)$  for its  $k$ -th element. Query block  $k$  attends to key/value block  $\pi(k)$ .

**The mask.** Define a  $T \times T$  masking matrix

$$M_{ij} = \begin{cases} 1 & \text{if } \pi\left(\left\lfloor \frac{in}{T} \right\rfloor\right) = \left\lfloor \frac{jn}{T} \right\rfloor \\ 0 & \text{otherwise} \end{cases} \quad i, j \in \{0, \dots, T-1\}.$$

Here  $\lfloor in/T \rfloor$  is the *block index* of token  $i$ : with  $T = 15$ ,  $n = 3$ , tokens 0–4 are in block 0, 5–9 in block 1, and 10–14 in block 2. Block attention is then

$$Z = \text{softmax}(QK^\top \odot M) \quad (\text{as written in the slides}).$$

### Common pitfall: how masking is actually implemented

Multiplying by a 0/1 mask *before* the softmax does not remove an entry, because  $\exp(0) = 1$  still gets probability mass. Real implementations add  $-\infty$  where  $M_{ij} = 0$ :

$$Z = \text{softmax}(QK^\top + (1 - M) \cdot (-\infty)),$$

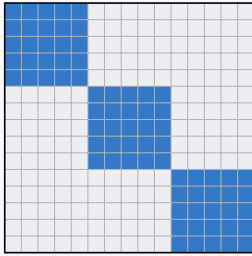
so masked entries become  $\exp(-\infty) = 0$ . Equivalently, you only *compute* the selected blocks, which is the whole point, and the unselected ones never exist.

### Worked example: $T = 15, n = 3$

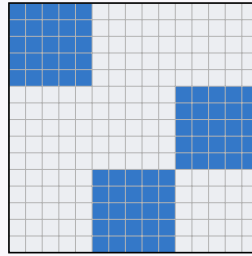
Write  $Q, K^\top$  and  $V$  as block matrices following the blocks in  $M$  (each block has 5 rows):

$$Q = (Q_0, Q_1, Q_2), \quad K = (K_{\pi(0)}, K_{\pi(1)}, K_{\pi(2)}), \quad V = (V_{\pi(0)}, V_{\pi(1)}, V_{\pi(2)}), \quad Q_i, K_i, V_i \in \mathbb{R}^{5 \times d}.$$

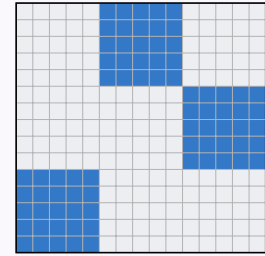
There are  $3! = 6$  ways of permuting the blocks. Two of them:



$\pi = (0, 1, 2)$  (identity)



$\pi = (0, 2, 1)$



$\pi = (1, 2, 0)$

With  $\pi = (0, 2, 1)$ : tokens 0–4 attend to tokens 0–4; tokens 5–9 attend to tokens **10–14**; tokens 10–14 attend to tokens **5–9**. Check one entry:  $i = 7, j = 12$  gives  $\lfloor 7 \cdot 3/15 \rfloor = 1, \pi(1) = 2$ , and  $\lfloor 12 \cdot 3/15 \rfloor = 2$ , so  $M_{7,12} = 1$ .

### 4.8.2 Computing block attention and its complexity

$$\text{Blockwise-Attention}(Q, K, V, M) = \begin{bmatrix} \text{softmax}\left(\frac{Q_0 K_{\pi(0)}^\top}{\sqrt{d}}\right) V_{\pi(0)} \\ \vdots \\ \text{softmax}\left(\frac{Q_{n-1} K_{\pi(n-1)}^\top}{\sqrt{d}}\right) V_{\pi(n-1)} \end{bmatrix} \in \mathbb{R}^{T \times d}.$$

#### Derivation: $\mathcal{O}(T^2 d/n)$

One block:  $Q_k K_{\pi(k)}^\top$  multiplies  $(\frac{T}{n} \times d)$  by  $(d \times \frac{T}{n})$ , which costs  $\frac{T}{n} \cdot \frac{T}{n} \cdot d$ . The product with  $V_{\pi(k)}$  costs the same. With  $n$  blocks:

$$\mathcal{O}\left(\frac{T}{n} \times \frac{T}{n} \times d \times n\right) = \boxed{\mathcal{O}\left(\frac{T^2 d}{n}\right)}.$$

**Blockwise multi-head attention.** The same idea extends to multi-head attention: **each head uses a different masking matrix**, derived from its own permutation. This is called *Blockwise Multi-Head Attention*. The union over heads covers far more of the  $T \times T$  grid than any single head.

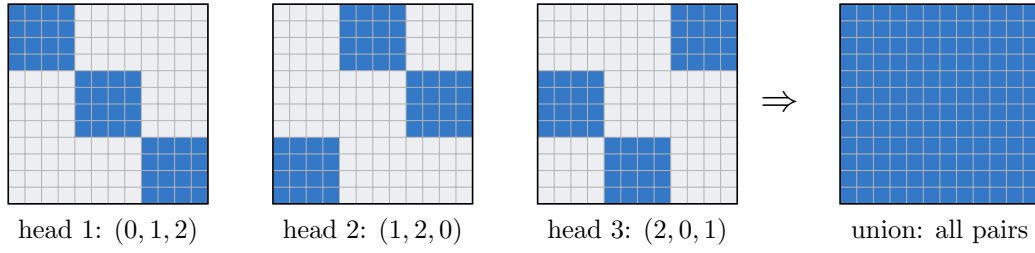
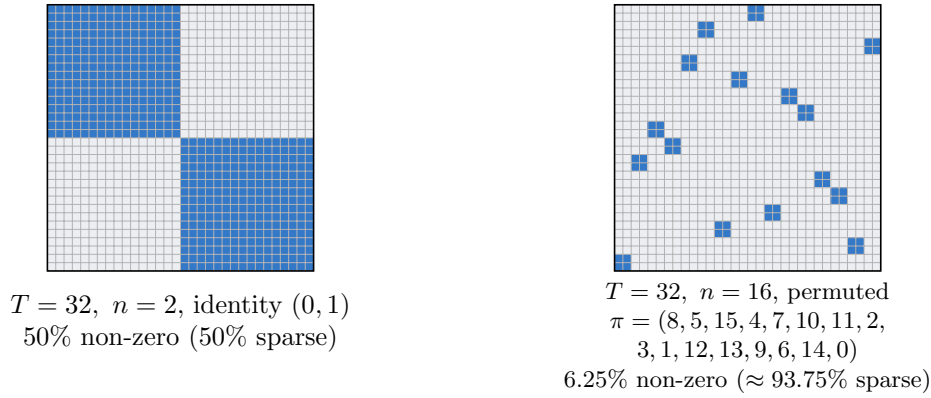


Figure 19: Three heads with three cyclic permutations together cover every (query-block, key-block) pair, while each head costs only  $1/3$  of full attention.

#### 4.8.3 What happens as $n$ grows?

The fraction of non-zero entries in  $M$  is

$$\frac{n(T/n)^2}{T^2} = \frac{1}{n}.$$



Larger  $n$  means sparser masks and less compute, but each query sees fewer keys.

#### 4.8.4 Capturing long-term dependency and performance

- Blockwise sparsity captures both **local** (identity permutation) and **long-distance** (non-identity permutations) dependencies in a memory-efficient way.
- Empirically, the **identity permutation is more important** than the others. This matches the BERT finding that neighbouring tokens matter most.
- Blockwise sparse attention **reduces training time significantly** for longer sequences.
- However, **perplexity increases as the number of blocks  $n$  increases**. This is expected, since local attention is only an approximation to full attention. The same trade-off applies to *all* local attention schemes.

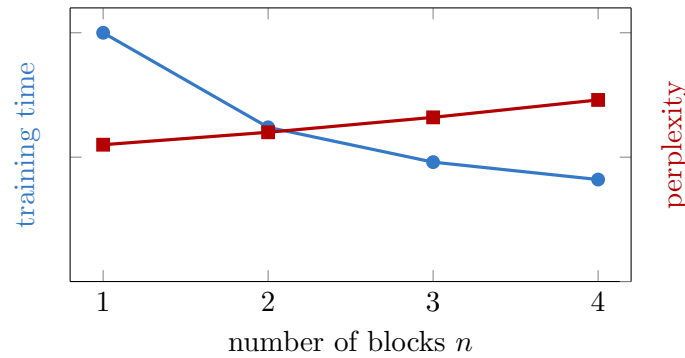


Figure 20: Qualitative trade-off (shape only, not measured values): more blocks means faster training but worse perplexity.

### Summary: sparse patterns

| Pattern                     | Cost                          | What it captures                           |
|-----------------------------|-------------------------------|--------------------------------------------|
| Full                        | $\mathcal{O}(T^2d)$           | everything                                 |
| Strided local (window $c$ ) | $\mathcal{O}(cTd)$            | neighbourhood                              |
| Dilated (window $c$ )       | $\mathcal{O}(cTd)$            | wider neighbourhood, same cost             |
| Global + local              | $\mathcal{O}((c + 2g)Td)$     | neighbourhood + task-critical tokens       |
| BigBird (glob.+rand.+local) | $\mathcal{O}((c + 2g + r)Td)$ | + short graph paths                        |
| Block ( $n$ blocks)         | $\mathcal{O}(T^2d/n)$         | in-block ( $\pi=\text{id}$ ) / cross-block |

## 4.9 Locality Sensitive Hashing attention (Reformer)

### 4.9.1 The idea: attention is approximately a nearest-neighbour search

Take two rows of an attention matrix. For a query  $q_i$ , only a few keys  $k_j$  get a high score, and the rest are close to zero after the softmax. **The attention matrix is mostly sparse.**

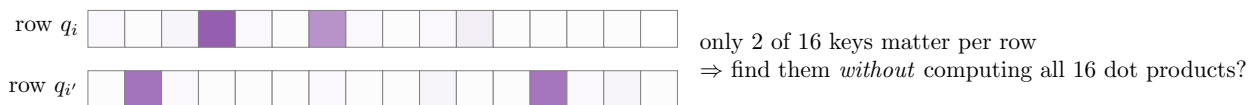


Figure 21: Two rows of a typical attention matrix. Darker cells mean higher probability.

- Can we find the high-scoring  $k_j$ 's **without computing the dot product with all the keys**?
- This is like approximating attention with a **K-Nearest-Neighbours (KNN)** search:  $q_i \cdot k_j$  is large when  $k_j$  is close to  $q_i$  in angle. If  $K$  has length 64K, for each  $q_i$  we could consider only a small subset of, say, the 32 or 64 closest keys.
- One of many ways to do this is **Locality Sensitive Hashing (LSH)**. This is the same family of techniques we met for *deduplicating* pre-training data (MinHash-LSH there; here, angular LSH on vectors).

### Intuition

An ordinary hash function scatters similar inputs to unrelated buckets on purpose. An LSH function does the opposite: **similar inputs collide on purpose**. Hash every query and key, then only compare vectors that landed in the same bucket. Each bucket is a small neighbourhood, so each bucket needs only a small attention computation.

### 4.9.2 The catch: shared query–key space

For this to work we need queries and keys to live in the *same* space and be hashed together, so the Reformer sets

$$Q = K \iff W_Q = W_K \text{ (shared-QK attention).}$$

If  $Q$  and  $K$  came from different projections, a query and its best key could land in different buckets and never meet.

#### Note: side effects of shared QK (from the Reformer paper)

- Keys are length-normalised, so the score depends on angle only.
- With  $Q = K$ , a token's dot product with itself is always the largest. So a token is *not* allowed to attend to itself unless it has no other target.
- The Reformer authors report that shared-QK does not noticeably hurt quality.

### 4.9.3 The hash function

We want a hashing scheme  $h(x)$  that assigns each vector  $x \in \mathbb{R}^d$  to a bucket so that **nearby vectors fall in the same bucket** and distant ones do not. Reformer uses *random rotations*:

$$h(x) = \arg \max \left( \text{concat}(xR, -xR) \right), \quad \begin{matrix} x \\ 1 \times d_k \end{matrix}, \quad \begin{matrix} R \\ d_k \times b/2 \end{matrix}$$

where  $b$  is the number of buckets and  $R$  is a random matrix.  $xR$  has  $b/2$  entries, and together with  $-xR$  there are  $b$  candidates. The bucket is the index of the largest.

#### Intuition: what the hash does geometrically

Project  $x$  onto  $b/2$  random directions  $r_1, \dots, r_{b/2}$  (the columns of  $R$ ) and their negatives. The largest projection tells you which of the  $b$  directions  $\{\pm r_\ell\}$  is *closest in angle* to  $x$ . The sphere is carved into  $b$  regions around those directions. Two vectors with a small angle between them almost always fall in the same region, and two vectors pointing in opposite directions almost never do.

### 4.9.4 A 2-D worked example

Assume  $d_k = d_q = 64$ . Take a query and a key,  $x = q_1$  and  $y = k_2$  with  $q_1, k_2 \in \mathbb{R}^{1 \times 64}$ , that are **far apart**, and project them onto a sphere (normalise). Set  $b = 4$  buckets, so  $R \in \mathbb{R}^{64 \times 2}$  (for illustration only two random directions).

Label the buckets by the index into  $\text{concat}(xR, -xR)$ :  $0 \leftrightarrow +r_1$ ,  $1 \leftrightarrow +r_2$ ,  $2 \leftrightarrow -r_1$ ,  $3 \leftrightarrow -r_2$ .

$$\begin{aligned} h(q_1) &= \arg \max(\text{concat}(\underbrace{0.5, 0.25}_{q_1 R}, \underbrace{-0.5, -0.25}_{-q_1 R})) = 0, \\ h(k_2) &= \arg \max(\text{concat}(\underbrace{-0.25, -0.5}_{k_2 R}, \underbrace{0.25, 0.5}_{-k_2 R})) = 3. \end{aligned}$$

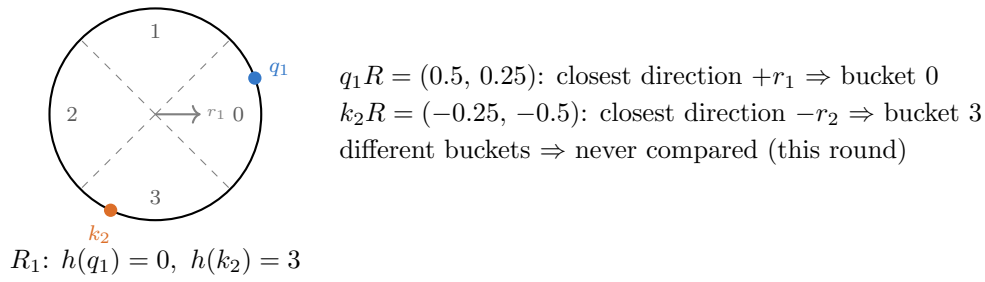


Figure 22: Four buckets on the circle (dashed lines are bucket boundaries, numbers are bucket IDs centred on  $+r_1, +r_2, -r_1, -r_2$ ).

Rather than draw each rotation, the slides record the bucket IDs obtained with three different random rotation matrices:

| $R_1$                 | $R_2$ | $R_3$ | fraction of rounds in the same bucket |                             |
|-----------------------|-------|-------|---------------------------------------|-----------------------------|
| <i>Far-apart pair</i> |       |       |                                       |                             |
| $q_1$                 | 0     | 2     | 1                                     | 1/3 $\Rightarrow$ not close |
| $k_2$                 | 3     | 2     | 0                                     |                             |
| <i>Close pair</i>     |       |       |                                       |                             |
| $q_1$                 | 0     | 2     | 1                                     | 3/3 $\Rightarrow$ close     |
| $k_2$                 | 0     | 2     | 1                                     |                             |

### Key idea

Recompute the hash with a few different rotation matrices. The *fraction of rounds in which two vectors share a bucket* estimates how close they are. Points that are close to each other end up in the same bucket (almost) every time.

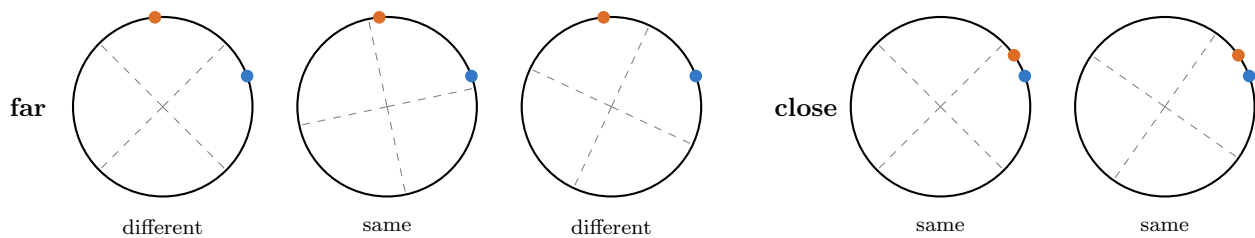


Figure 23: Left three: a pair about  $75^\circ$  apart (blue, orange) under three random rotations of the bucket boundaries collides in 1 of 3 rounds. Right: a close pair collides in every round unless a boundary happens to fall between them, which is rare.

### Note: why this works — collision probability

For random-hyperplane LSH (the  $b = 2$  case:  $h(x) = \text{sign}(x \cdot r)$ ), the probability that two unit vectors separated by angle  $\theta$  collide is exactly

$$\Pr[h(x) = h(y)] = 1 - \frac{\theta}{\pi}.$$

Small angle means high collision probability. Since  $q \cdot k = \cos \theta$  for unit vectors, high attention score means high collision probability. The random-rotation hash with  $b$  buckets makes this sharper.

#### 4.9.5 The full LSH attention procedure

1. **Hash all queries (keys) in parallel:** one matrix multiplication  $QR$  gives every token's projections, then an argmax per row. Repeat with  $n_R$  independent rotations if more robustness is needed.
2. **Sort** the tokens by bucket number, and within each bucket by sequence position.
3. **Chunk** the sorted sequence into equal-sized chunks, so the work parallelises nicely even when buckets have uneven sizes.
4. **Attend within buckets:** each query attends only to keys in the same bucket (in its own chunk and the previous chunk). In a causal LM (CLM) the causal mask still applies. For example, if  $q_1, q_3, q_{16}$  share bucket 0, then  $q_3$  attends to  $q_1$ , and  $q_{16}$  attends to  $q_3$  and  $q_1$ .

| Query    | hash by $R_1$ | hash by $R_2$ |                                             | Query    | $R_1$    | $R_2$    |
|----------|---------------|---------------|---------------------------------------------|----------|----------|----------|
| $q_1$    | 0             | 2             | sort by bucket, then position $\rightarrow$ | $q_1$    | 0        | 2        |
| $q_2$    | 3             | 2             |                                             | $q_3$    | 0        | 2        |
| $q_3$    | 0             | 2             |                                             | $q_{16}$ | 0        | 2        |
| $\vdots$ | $\vdots$      | $\vdots$      |                                             | $\vdots$ | $\vdots$ | $\vdots$ |
| $q_{16}$ | 0             | 2             |                                             | $q_2$    | 3        | 2        |
|          |               |               |                                             |          |          |          |

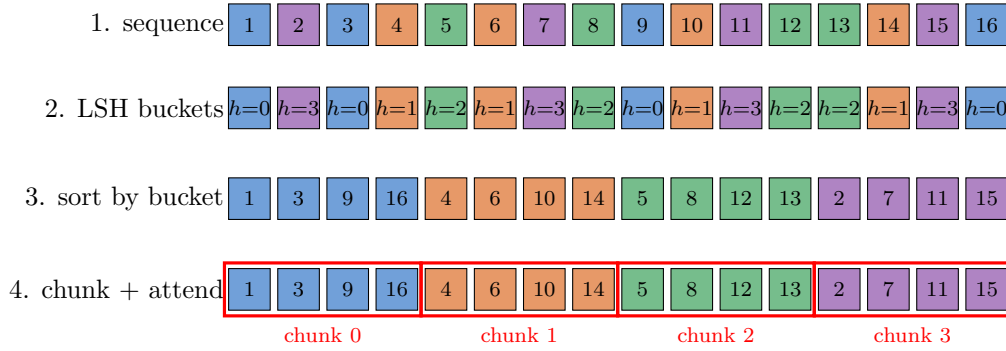


Figure 24: The LSH attention pipeline on 16 tokens with 4 buckets (colours). After sorting, same-bucket tokens are contiguous, so attention becomes a block-diagonal computation over small chunks.

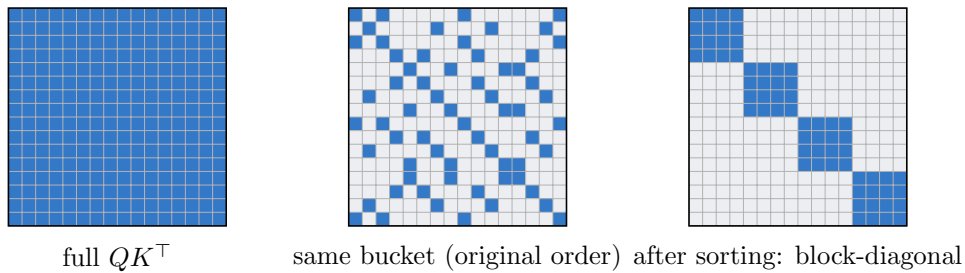


Figure 25: The same-bucket attention pattern is scattered in the original order but becomes block-diagonal after sorting by bucket. That block-diagonal shape is what makes LSH attention efficient on hardware.

#### 4.9.6 Complexity

Hashing costs  $\mathcal{O}(Tdb)$ , and sorting costs  $\mathcal{O}(T \log T)$ . Each query then attends to a constant-size chunk. For each query, finding the approximate nearest neighbours takes  $\mathcal{O}(\log T)$  time (the sorting

cost amortised per query), so the overall complexity is

$$\mathcal{O}(T \log T).$$

#### Note: other Reformer ingredients (for context)

The Reformer paper combines LSH attention with two memory tricks: **reversible residual layers**, which recompute activations in the backward pass instead of storing one copy per layer, and **chunked feed-forward layers**. These reduce memory; LSH reduces the attention cost. Multi-round hashing ( $n_R$  rounds) lowers the chance that two similar vectors are separated, at  $n_R$  times the cost.

#### Summary: LSH attention

- Attention is mostly sparse, so treat it as approximate nearest-neighbour search.
- Shared QK ( $W_Q = W_K$ ) is needed so queries and keys are hashed into one space.
- $h(x) = \arg \max(\text{concat}(xR, -xR))$  with  $R \in \mathbb{R}^{d_k \times b/2}$ , repeated over several random  $R$ .
- Hash, sort by (bucket, position), chunk, attend within bucket:  $\mathcal{O}(T \log T)$ .

## 4.10 Self-attention is low rank (Linformer)

### 4.10.1 Where we stand

The methods so far **localise or sparsify** the attention matrix in different ways. Their theoretical time complexity is sub-quadratic,  $\mathcal{O}(cTd)$ , where  $c$  is a constant (set by hand, like a window size) or a function of  $T$  (like  $\log T$  for LSH).

**Can we reduce it to  $\mathcal{O}(Td)$ ?** Trivially yes: let every token attend only to itself ( $c = 1$ ). That is useless, because no context is captured. We need a smarter route to linear cost.

### 4.10.2 The empirical observation

The self-attention matrix is **essentially low-rank**, especially in the top layers.

- Take a RoBERTa model trained on Wiki103 (MLM task) and on a classification task. Each attention matrix is  $A = \text{softmax}(QK^\top / \sqrt{d}) \in \mathbb{R}^{512 \times 512}$ .
- Apply **SVD** to  $A$  for different layers and heads, and plot the *normalised cumulative singular value*, averaged over 10K sentences.
- **Question:** for  $A$  to be low-rank, what distribution should its 512 singular values follow?  
**Answer:** a **long-tailed** distribution: a few large singular values carry most of the “energy” and the rest are tiny.
- This is what was observed, so they concluded that  $A$  is (approximately) low-rank.

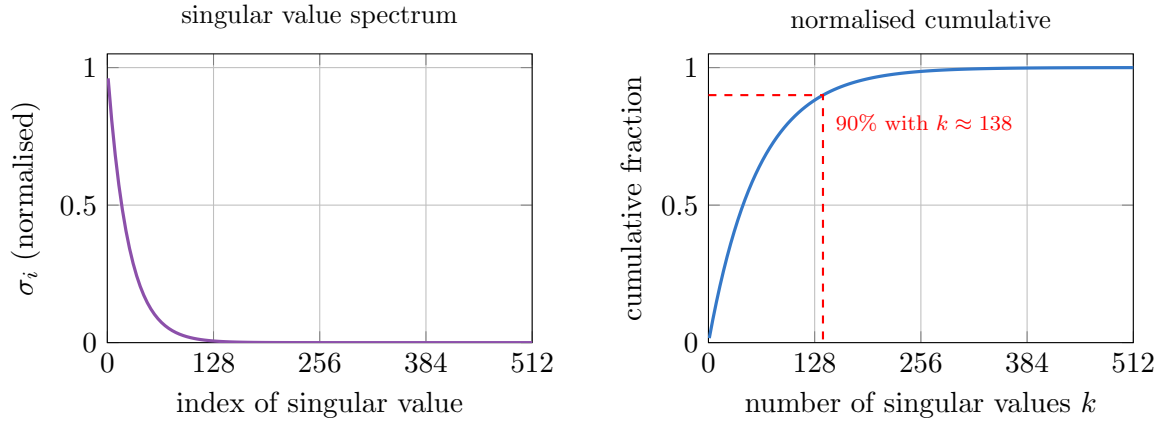


Figure 26: Illustrative shape of a long-tailed spectrum (not the paper’s measured values). Most of the matrix is explained by a small number  $k \ll T$  of directions.

#### Intuition: what low rank means for attention

A rank- $k$  matrix  $A \in \mathbb{R}^{T \times T}$  can be written as  $A = UV^\top$  with  $U, V \in \mathbb{R}^{T \times k}$ . Its  $T$  rows are combinations of only  $k$  basic patterns. In attention terms, although there are  $T$  keys, the queries only “care” about  $k$  different mixtures of them. So summarise the  $T$  keys into  $k$  representative keys and attend to those.

#### 4.10.3 The Linformer construction

The observation says a low-rank approximation of  $A$  exists, and the network can **learn** it. Introduce two learnable linear projection matrices

$$\begin{matrix} E \\ k \times T \end{matrix}, \quad \begin{matrix} F \\ k \times T \end{matrix}$$

that compress the *sequence length* of keys and values from  $T$  to  $k$ :

$$\begin{aligned} EK &= \begin{matrix} E & K \\ k \times d & k \times T & T \times d \end{matrix}, & FV &= \begin{matrix} F & V \\ k \times d & k \times T & T \times d \end{matrix}, \\ \bar{A} &= \text{softmax} \left( \frac{Q (EK)^\top}{\sqrt{d}} \right), \\ O &= \bar{A} FV. \end{aligned}$$

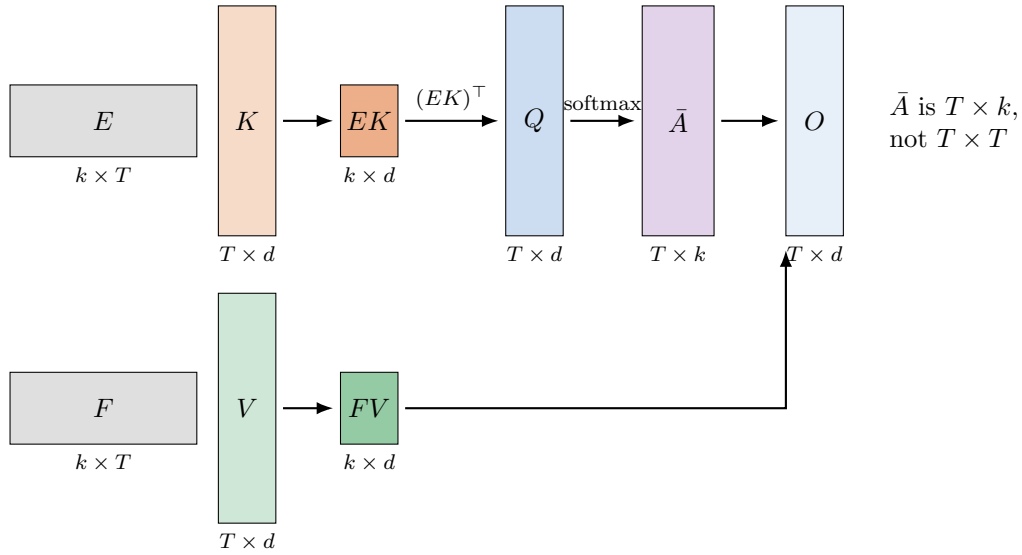


Figure 27: Linformer:  $E$  and  $F$  compress the length dimension of  $K$  and  $V$  from  $T$  to  $k$ . The score matrix shrinks from  $T \times T$  to  $T \times k$ .

#### Derivation: Linformer complexity

$$EK, FV : (k \times T)(T \times d) \rightarrow \mathcal{O}(kTd)$$

$$Q(EK)^\top : (T \times d)(d \times k) \rightarrow \mathcal{O}(Tdk)$$

$$\text{softmax} : T \text{ rows of length } k \rightarrow \mathcal{O}(Tk)$$

$$\bar{A}(FV) : (T \times k)(k \times d) \rightarrow \mathcal{O}(Tkd)$$

Total:  $\mathcal{O}(kTd)$ , **linear in  $T$**  for a fixed rank  $k$ . Memory for the score matrix drops from  $T^2$  to  $Tk$ .

The rank  $k$  is a hyperparameter set according to the tolerated approximation error  $\epsilon$  (the paper proves a suitable  $k$  exists that depends on  $d$  and  $\epsilon$ , not on  $T$ ).

#### Note: limitations worth knowing

- $E$  and  $F$  have shape  $k \times T$ , so the sequence length  $T$  is baked into the parameters: the model handles a fixed maximum length.
- Mixing across positions with  $E$  breaks causality, since a compressed key can contain future tokens. Plain Linformer therefore suits encoders (BERT-like) better than autoregressive decoders.
- Parameter sharing (e.g.  $E = F$ , or one  $E$  shared across heads and layers) works almost as well and saves parameters.

## 4.11 Kernel methods (Performer)

### 4.11.1 Rewriting softmax attention

All we need is an attention matrix whose rows sum to one. Look at what the softmax does: the exponential takes a query and a key and outputs a **positive number**:

$$\text{Atten} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right), \quad A(i, j) = \text{SM}(q_i, k_j) = \exp\left(\frac{q_i^\top k_j}{\sqrt{d}}\right).$$

Separate the exponentiation from the normalisation:

$$\text{Atten} = D^{-1}A, \quad A_{T \times T} = \exp(QK^\top / \sqrt{d}), \quad D_{T \times T} = \text{diag}(A\mathbf{1}_T)$$

where  $\mathbf{1}_T$  is the all-ones vector of length  $T$ , so  $A\mathbf{1}_T$  holds the row sums and  $D^{-1}$  divides each row by its sum.

### 4.11.2 Generalising to a kernel

So  $A(i, j) = K(q_i, k_j)$  for some function  $K : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}_+$ . Such a  $K$  is called a **kernel**. The Performer writes it as an expectation over a random feature map:

$$K(q_i, k_j) = \mathbb{E}[\phi(q_i)^\top \phi(k_j)], \quad \phi : \mathbb{R}^d \rightarrow \mathbb{R}_+^r, \quad r > 0$$

where  $\phi$  is a **randomised feature mapping** and  $\mathbb{E}$  is the expectation over its randomness.

#### Note

$r > 0$ , and it is *not* necessary that  $r < d$ . For example, with  $T = 64\text{K}$  and  $d = 4\text{K}$ ,  $r = 8\text{K}$  is also reasonable. What matters is  $r \ll T$ .

#### Key idea: why a feature map gives linear time

If  $A \approx Q'K'^\top$  with  $Q', K' \in \mathbb{R}^{T \times r}$ , we never need to form the  $T \times T$  matrix. Matrix multiplication is associative:

$$\underbrace{(Q'K'^\top)}_{T \times T} V = Q' \underbrace{(K'^\top V)}_{r \times d}.$$

The left side costs  $\mathcal{O}(T^2r + T^2d)$  and the right side costs  $\mathcal{O}(Trd)$ .

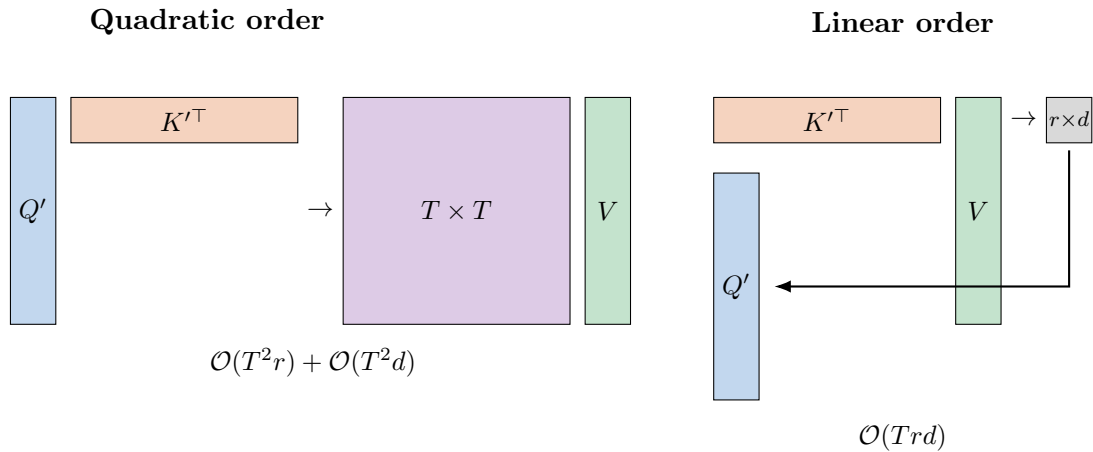


Figure 28: Performer's trick: multiply  $K'^\top V$  first. The  $T \times T$  matrix is never formed. (The Performer paper draws this with  $L$  for sequence length;  $L = T$ .)

### 4.11.3 How do we compute $\phi(x)$ ?

The general form of the random feature map is

$$\phi(x) = \frac{h(x)}{\sqrt{r}} \left( f_1(w_1^\top x), \dots, f_1(w_r^\top x), \dots, f_l(w_1^\top x), \dots, f_l(w_r^\top x) \right),$$

with  $f_1, \dots, f_l : \mathbb{R} \rightarrow \mathbb{R}$ ,  $h : \mathbb{R}^d \rightarrow \mathbb{R}$ , and random vectors  $w_1, \dots, w_r \in \mathbb{R}^d$  sampled i.i.d. from  $\mathcal{N}(\mathbf{0}_d, I_d)$ .

$w_i^\top x$  is a *random projection*: the vector  $x$  projected onto a random direction  $w_i$ .

The Performer's theoretically motivated choice (one of many possible):

$$l = 1, \quad f_1 = \exp, \quad h(x) = \exp \left( -\frac{\|x\|^2}{2} \right),$$

which gives the **positive random features**

$$\phi(x) = \frac{1}{\sqrt{r}} \exp \left( -\frac{\|x\|^2}{2} \right) \left( \exp(w_1^\top x), \dots, \exp(w_r^\top x) \right) \in \mathbb{R}_+^r.$$

**Derivation: why  $\mathbb{E}[\phi(q)^\top \phi(k)] = \exp(q^\top k)$**

**Step 1 (Gaussian moment generating function).** For  $w \sim \mathcal{N}(\mathbf{0}, I_d)$  and a fixed  $u \in \mathbb{R}^d$ ,  $w^\top u \sim \mathcal{N}(0, \|u\|^2)$ , so

$$\mathbb{E}[\exp(w^\top u)] = \exp \left( \frac{\|u\|^2}{2} \right).$$

**Step 2 (one feature).**

$$\mathbb{E}[\exp(w^\top q) \exp(w^\top k)] = \mathbb{E}[\exp(w^\top (q + k))] = \exp \left( \frac{\|q + k\|^2}{2} \right) = \exp \left( \frac{\|q\|^2}{2} + \frac{\|k\|^2}{2} + q^\top k \right).$$

**Step 3 (multiply by the  $h$  factors).**

$$\exp \left( -\frac{\|q\|^2}{2} \right) \exp \left( -\frac{\|k\|^2}{2} \right) \cdot \exp \left( \frac{\|q\|^2}{2} + \frac{\|k\|^2}{2} + q^\top k \right) = \exp(q^\top k).$$

**Step 4 (average over  $r$  features).**

$$\phi(q)^\top \phi(k) = \frac{1}{r} \sum_{i=1}^r \exp \left( -\frac{\|q\|^2}{2} \right) \exp(w_i^\top q) \exp \left( -\frac{\|k\|^2}{2} \right) \exp(w_i^\top k),$$

which is an average of  $r$  i.i.d. unbiased estimates, so  $\mathbb{E}[\phi(q)^\top \phi(k)] = \exp(q^\top k)$ , with variance falling as  $1/r$ .

**Step 5 (the  $\sqrt{d}$  scaling).** To get  $\exp(q^\top k / \sqrt{d})$ , feed  $q/d^{1/4}$  and  $k/d^{1/4}$  into  $\phi$ .

**Intuition: why positive features**

Earlier random-feature methods used sin / cos features. Those can give *negative* estimates of a quantity that must be positive, and they are very noisy when the true value is near zero, which is exactly where most attention entries sit.  $\exp(\cdot)$  features are always positive, so the estimated row sums (the normaliser  $D$ ) are positive and training stays stable.

#### 4.11.4 Vectorised form

Collect the  $w_i$  in a matrix. In column convention ( $X \in \mathbb{R}^{d \times T}$  holds tokens as columns,  $W \in \mathbb{R}^{d \times r}$ ):

$$\phi(X) = \frac{1}{\sqrt{r}} \exp \left( W^\top X - \frac{\|X\|^2}{2} \right),$$

where  $\|X\|^2$  holds the squared  $L_2$  norms of the columns of  $X$  (broadcast along rows). In the row convention used for  $Q$  ( $Q \in \mathbb{R}^{T \times d}$ ,  $W \in \mathbb{R}^{r \times d}$ ):

$$Q' = \frac{1}{\sqrt{r}} \exp \left( Q W^\top - \frac{\|Q\|^2}{2} \right)$$

with  $\|Q\|^2 \in \mathbb{R}^T$  the squared norms of the rows of  $Q$  (broadcast along columns). Similarly  $K'$ .

The approximated attention entry and matrix are

$$\hat{A}(i, j) = \widehat{\text{SM}}(q_i, k_j) = \frac{1}{r} \exp \left( q_i^\top W^\top - \frac{\|q_i\|^2}{2} \right) \exp \left( k_j^\top W^\top - \frac{\|k_j\|^2}{2} \right)^\top, \quad \boxed{\hat{A} = Q' K'^\top}.$$

The full normalised output, computed in linear time:

$$\widehat{\text{Atten}} V = \hat{D}^{-1} (Q' (K'^\top V)), \quad \hat{D} = \text{diag} (Q' (K'^\top \mathbf{1}_T)).$$

The normaliser also costs only  $\mathcal{O}(Tr)$ : compute  $K'^\top \mathbf{1}_T \in \mathbb{R}^r$  first.

#### 4.11.5 Orthogonal random features

The matrix  $W$  is **orthogonalised with the Gram–Schmidt procedure**. Orthogonal  $w_i$ 's explore different directions instead of randomly overlapping ones, which **reduces the variance** of the softmax estimator for any dimensionality  $d$ . Exact orthogonality requires  $r \leq d$ , since at most  $d$  vectors in  $\mathbb{R}^d$  can be mutually orthogonal. For  $r > d$ , stack independent orthogonal blocks.

#### 4.11.6 Complexity

Computing  $Q'$  and  $K'$ :  $(T \times d)(d \times r) \Rightarrow \mathcal{O}(rTd)$ . Then  $K'^\top V$ :  $\mathcal{O}(rTd)$ , and  $Q'(K'^\top V)$ :  $\mathcal{O}(Trd)$ . So the time complexity is

$$\boxed{\mathcal{O}(rTd)}, \quad \text{linear in } T.$$

#### Note: causal (decoder) Performer

For autoregressive models, query  $i$  may only use keys  $\leq i$ . Replace  $K'^\top V$  by *prefix sums*  $S_i = \sum_{j \leq i} \phi(k_j) v_j^\top \in \mathbb{R}^{r \times d}$ , updated one token at a time. The cost stays linear, and it gives an RNN-like view of attention with a constant-size state.

### Summary: low-rank and kernel methods

|                 | Linformer                                                   | Performer                                                  |
|-----------------|-------------------------------------------------------------|------------------------------------------------------------|
| Core assumption | $A$ is low-rank                                             | $\exp(q^\top k)$ is a kernel with random features          |
| Mechanism       | learn $E, F \in \mathbb{R}^{k \times T}$ to compress $K, V$ | $\phi$ maps $q, k$ to $\mathbb{R}_+^r$ ; use associativity |
| Score matrix    | $T \times k$                                                | never formed ( $T \times r$ features)                      |
| Complexity      | $\mathcal{O}(kTd)$                                          | $\mathcal{O}(rTd)$                                         |
| Exact?          | approximation                                               | unbiased approximation                                     |
| Causal?         | not directly                                                | yes, via prefix sums                                       |

#### 4.12 Reduced FLOPs $\neq$ reduced wall-clock time

Every approach in this module **approximates** full attention to reduce FLOPs. In practice, this often does **not** translate into a wall-clock speed-up.

Suppose training a model with compute requirement  $C \propto T^2$  FLOPs takes one day. Reducing the compute to  $C \propto T^2/2$  FLOPs does **not** reduce the training time to half a day.

#### Intuition

Think of a chef who can chop a vegetable in one second but has to walk to a distant storeroom for each one. Buying a faster knife (fewer FLOPs) barely helps, because the walking (memory traffic) dominates. Sparse and irregular attention patterns can make the walking *worse*, since scattered memory accesses defeat the GPU's preference for large, contiguous, dense blocks.

So the problem is not in the algorithm but in how it uses the **hardware**. This leads to Module 4.

## 5 Module 4: Know your hardware to get wall-clock speed

### 5.1 FlashAttention: exact attention with IO-awareness

All Transformer language models are trained on GPUs, and it was observed that the standard implementation of multi-head attention did **not efficiently utilise** them. The training time turned out to be **memory-bound, not compute-bound**: the GPU spends more time moving data than doing arithmetic.

#### Key idea

FlashAttention computes **exact** attention (no approximation) with the same  $\mathcal{O}(T^2d)$  FLOPs, but with far fewer reads and writes to slow GPU memory. It is faster in wall-clock time and uses  $\mathcal{O}(T)$  instead of  $\mathcal{O}(T^2)$  extra memory.

### 5.2 A toy example: every operation is read, compute, write

Consider computing

$$c = ab, \quad d = \sqrt{c}.$$

Executed naively, each operation is a separate step: read inputs from main memory, compute in registers, and write the result back to main memory.

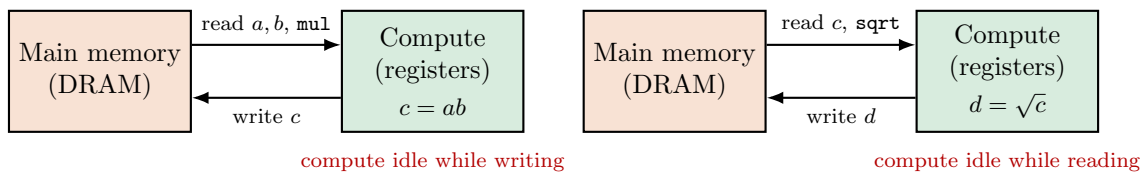


Figure 29: Unfused execution: 2 reads, 2 writes, 2 compute operations.  $c$  goes to main memory only to be read straight back.

There are **two reads, two writes and two compute operations**, and compute sits idle while results are written back. Typically, after executing an operation, the result is written back to main memory. How do we skip this redundant writing and reading when we *know*  $c$  is used by the very next operation?

### 5.3 Kernel fusion

**Fuse** the sequence of operations into a single GPU kernel. Keep the intermediate  $c$  in registers/-cache, take its square root there, and write only the final result back.

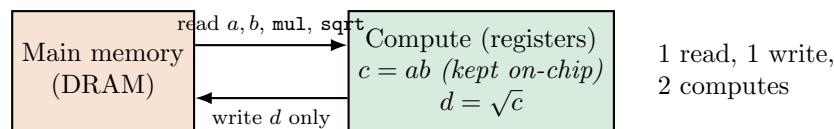


Figure 30: Fused kernel: the intermediate result never leaves the chip.

#### Key idea

Kernel fusion keeps the compute units busy by telling the GPU that intermediate results need not be written back to main memory.

## 5.4 Naive multi-head attention is memory-bound

A standard (e.g. PyTorch) implementation of attention runs a chain of separate kernels:

$$\begin{aligned} QK^\top &\rightarrow QK^\top + M \rightarrow \text{softmax}(QK^\top + M) \\ &\rightarrow \text{drop}(\text{softmax}(QK^\top + M)) \rightarrow \text{drop}(\text{softmax}(QK^\top + M)) V. \end{aligned}$$

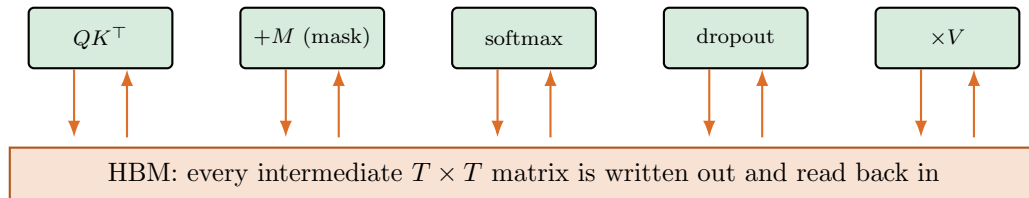


Figure 31: Naive attention: every step writes its output to memory and the next step reads it back.

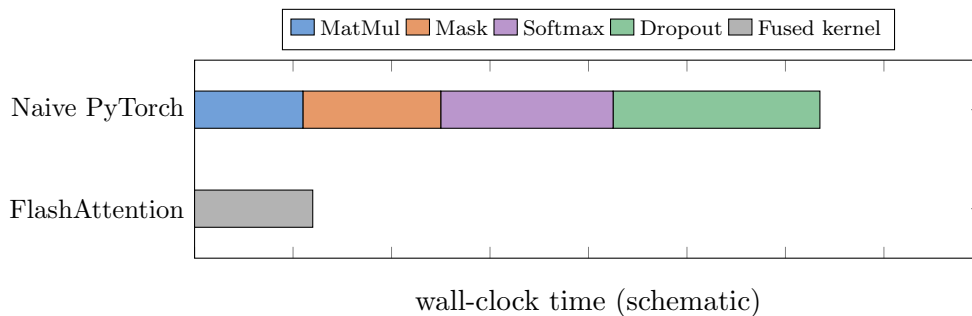


Figure 32: Schematic of the slide's timing figure (proportions indicative only). In the naive version **dropout, softmax and masking take longer than the matrix multiplications**. One fused FlashAttention kernel is much faster overall.

**Surprise:** dropout, softmax and masking take more time than the matmul, although they do far fewer FLOPs.

### Intuition: arithmetic intensity

The *arithmetic intensity* of an operation is FLOPs per byte moved. A matmul of  $n \times n$  matrices does  $\sim 2n^3$  FLOPs on  $\sim 3n^2$  numbers, so its intensity grows with  $n$ : **compute-bound**. An element-wise operation (mask, softmax, dropout, activation, normalisation) does about one FLOP per number read and written: **memory-bound**. An A100 can do roughly 300 TFLOP/s but move only about 1.5–2 TB/s from HBM. To keep the arithmetic units busy you need on the order of 100+ FLOPs per byte, which element-wise operations come nowhere near.

In general, **all element-wise operations are memory-bound**, and the general remedy is **kernel fusion**.

### Common pitfall: the catch

We cannot simply fuse everything and discard intermediates, because the **backward pass needs**  $QK^\top$  (and the softmax output  $P$ ) to compute gradients. The naive implementation stores these  $T \times T$  matrices, which costs  $\mathcal{O}(T^2)$  memory and  $\mathcal{O}(T^2)$  HBM traffic. FlashAttention's answer is *recomputation* (Section 5.13).

## 5.5 The objective, precisely

$$S = QK^\top \in \mathbb{R}^{N \times N}, \quad P = \text{softmax}(S) \in \mathbb{R}^{N \times N}, \quad O = PV \in \mathbb{R}^{N \times d}.$$

(From here on we follow the FlashAttention paper and write  $N$  for the sequence length.) Standard attention is memory-bound.

**Objective:** given  $Q, K, V$  in HBM (High Bandwidth Memory), compute  $O$  and write it to HBM **using only one write operation**, never materialising  $S$  or  $P$  in HBM.

We cannot load the entire  $Q, K, V$  into fast memory at once. We must divide them into **blocks**. Why? Because of the memory hierarchy.

## 5.6 GPU memory hierarchy

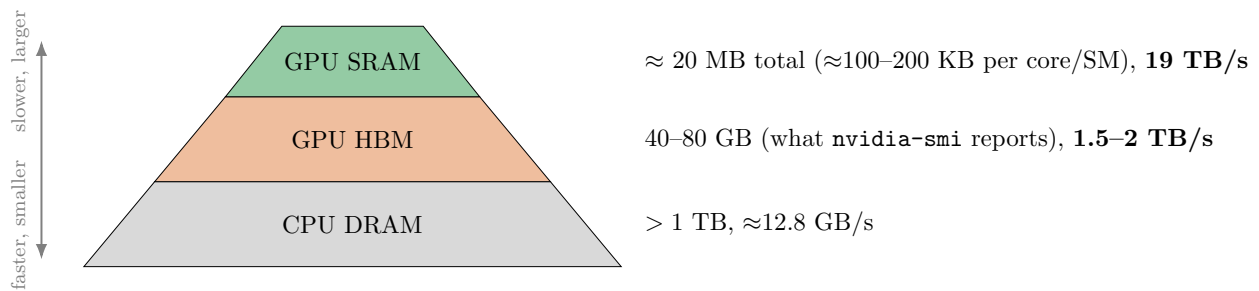


Figure 33: Memory hierarchy (A100-class GPU figures from the FlashAttention paper). SRAM is about  $10\times$  faster than HBM but about  $1000\times$  smaller.

- **HBM** (High Bandwidth Memory) is what you see with `nvidia-smi`. Data, model parameters and all intermediate results are loaded from and stored here.
- **SRAM** is small (about 20 MB, distributed across cores at about 100 KB each) with very high bandwidth (19 TB/s).

### Worked example: why $Q$ and $K$ do not fit

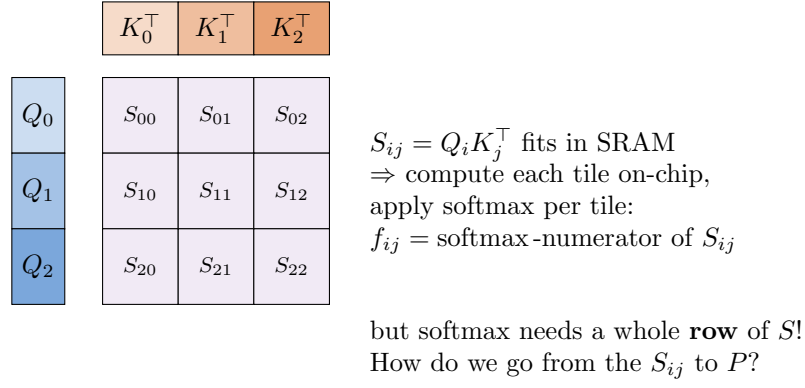
Sequence length  $N = 1024$ , head dimension  $d = 64$ , 4 bytes (fp32) per element. Storing  $Q$  and  $K$ :

$$2 \times 1024 \times 64 \times 4 = 524,288 \text{ bytes} \approx 524 \text{ KB} \gg 100 \text{ KB}.$$

So the entire  $Q$  and  $K$  cannot fit into one core's SRAM, let alone the  $N \times N = 4 \text{ MB}$  score matrix.

## 5.7 Idea 1: tiling

Divide  $Q, K, V$  (which live in HBM) into small blocks that fit into SRAM (size  $M \approx 100 \text{ KB}$ ) and compute attention block by block.

Figure 34: Tiling  $S = QK^\top$  into blocks.

We can compute  $S_{ij} = Q_i K_j^\top$  for one tile and exponentiate it, giving an intermediate  $f_{ij}$ . But the softmax normaliser needs *all* elements of a row of  $S$ . The plan is to assemble  $P$  as a rescaled concatenation of tile results:

$$P = \text{Concat}(\beta_0 f_{00}, \beta_1 f_{01}, \dots, \beta_8 f_{22}) \text{ (then normalise),}$$

where  $f_{ij}$  is the intermediate (exponentiated) tile and  $\beta$  is a correction weight per tile. How do we compute the weights  $\beta$ ? This is the **online softmax** trick.

## 5.8 Safe softmax

First a numerically stable variant, **safe softmax**. Subtract the maximum of the vector from all its elements before exponentiating:

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}} = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}, \quad m = \max(x).$$

**Derivation: safe softmax is identical**

$$\frac{e^{x_i - m}}{\sum_j e^{x_j - m}} = \frac{e^{x_i} e^{-m}}{e^{-m} \sum_j e^{x_j}} = \frac{e^{x_i}}{\sum_j e^{x_j}}.$$

After subtracting  $m$ , every exponent is  $\leq 0$ , so  $e^{x_i - m} \in (0, 1]$  and cannot overflow. (In fp16,  $e^{12} \approx 1.6 \times 10^5$  already exceeds the largest representable value of about 65,504.) The exponential is more accurate for non-positive inputs.

## 5.9 Online softmax: a worked example

Consider a sequence  $x$  split into two consecutive sub-sequences  $x_A$  and  $x_B$ :

$$x_A = [-1.02, -1.09, 0.24, 0.87, -1.32], \quad x_B = [-0.60, -5.04, -2.00, 1.03, -0.14].$$

**Split: process each block independently.**

|                       | elements |       |       |      |       | statistics                        |
|-----------------------|----------|-------|-------|------|-------|-----------------------------------|
| $x_A$                 | -1.02    | -1.09 | 0.24  | 0.87 | -1.32 | $m_1 = \max(x_A) = 0.87$          |
| $f_1 = e^{x_A - m_1}$ | 0.15     | 0.14  | 0.53  | 1.00 | 0.11  | $l_1 = \sum e^{x_A - m_1} = 1.94$ |
| $x_B$                 | -0.60    | -5.04 | -2.00 | 1.03 | -0.14 | $m_2 = \max(x_B) = 1.03$          |
| $f_2 = e^{x_B - m_2}$ | 0.20     | 0.00  | 0.05  | 1.00 | 0.31  | $l_2 = \sum e^{x_B - m_2} = 1.56$ |

**Combine: fix the stale maxima.**

$$\begin{aligned}
 m &= \max(m_1, m_2) = 1.03, \\
 \beta_1 &= e^{m_1 - m} = e^{0.87 - 1.03} = e^{-0.16} = 0.85, \quad \beta_2 = e^{m_2 - m} = e^0 = 1, \\
 \beta_1 f_1 &= [0.13, 0.12, 0.45, 0.85, 0.10], \quad \beta_2 f_2 = [0.20, 0.00, 0.05, 1.00, 0.31], \\
 f &= [\beta_1 f_1 : \beta_2 f_2] \quad (\text{concatenation, length } 10), \\
 l &= \beta_1 l_1 + \beta_2 l_2 = 0.85 \times 1.94 + 1.56 = 3.21, \\
 P &= f/l = [0.04, 0.04, 0.14, 0.27, 0.03, 0.06, 0.00, 0.01, 0.31, 0.10].
 \end{aligned}$$

This matches the softmax of the full 10-element vector computed directly (unrounded: 0.040, 0.037, 0.142, 0.266, 0.030, 0.061, 0.001, 0.015, 0.312, 0.097).

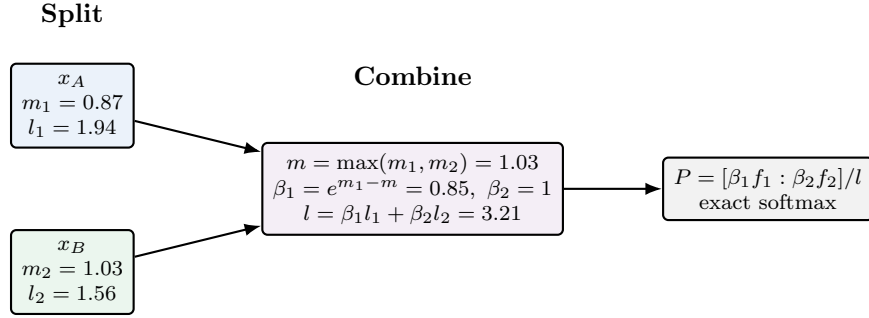


Figure 35: Online softmax: each block keeps only its local max  $m$  and local sum  $l$ . Blocks are combined exactly by rescaling with  $\beta = e^{m_{\text{old}} - m_{\text{new}}}$ .

### Key idea

To recover  $P$  from the per-block  $f_i$  we only need to keep track of the running statistics  $m_i$  (max) and  $l_i$  (sum of exponentials). These are two numbers per row, not a whole row of  $S$ .

### Derivation: general online softmax recurrence

Process a row of scores in blocks  $x^{(1)}, x^{(2)}, \dots$ . Maintain  $m^{(j)} = \max$  of everything seen so far and  $l^{(j)} = \sum_{\text{seen}} e^{x - m^{(j)}}$ . For a new block with local statistics  $\tilde{m} = \max x^{(j+1)}$  and  $\tilde{l} = \sum e^{x^{(j+1)} - \tilde{m}}$ :

$$\begin{aligned}
 m^{(j+1)} &= \max(m^{(j)}, \tilde{m}), \\
 l^{(j+1)} &= e^{m^{(j)} - m^{(j+1)}} l^{(j)} + e^{\tilde{m} - m^{(j+1)}} \tilde{l}.
 \end{aligned}$$

**Proof.**  $e^{m^{(j)} - m^{(j+1)}} l^{(j)} = \sum_{\text{old}} e^{x - m^{(j)}} e^{m^{(j)} - m^{(j+1)}} = \sum_{\text{old}} e^{x - m^{(j+1)}}$ , and the same holds for the new block. Adding gives  $\sum_{\text{all}} e^{x - m^{(j+1)}}$ .  $\square$

### Derivation: rescaling the partial output

We need  $O$ , not  $P$ . For one query row, after key blocks  $1..j$  define the *normalised partial output*

$$o^{(j)} = \frac{1}{l^{(j)}} \sum_{j' \leq j} \sum_{t \in \text{block } j'} e^{s_t - m^{(j)}} v_t.$$

When block  $j+1$  arrives with local scores  $\tilde{s}$ , local max  $\tilde{m}$ , local sum  $\tilde{l}$  and values  $V_{j+1}$ :

$$o^{(j+1)} = \frac{1}{l^{(j+1)}} \left( l^{(j)} e^{m^{(j)} - m^{(j+1)}} o^{(j)} + e^{\tilde{m} - m^{(j+1)}} \tilde{p} V_{j+1} \right), \quad \tilde{p} = e^{\tilde{s} - \tilde{m}}.$$

Multiplying  $o^{(j)}$  by  $l^{(j)}$  undoes the old normalisation, multiplying by  $e^{m^{(j)}-m^{(j+1)}}$  updates the old max to the new one, and dividing by  $l^{(j+1)}$  applies the new normaliser. After the last block,  $o = \text{softmax}(s) V$  **exactly**. In matrix form for a query tile  $i$  (the update used in the algorithm):

$$O_i \leftarrow \text{diag}(l_i^{\text{new}})^{-1} \left( \text{diag}(l_i) e^{m_i - m_i^{\text{new}}} O_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{P}_{ij} V_j \right).$$

### 5.10 The FlashAttention algorithm (forward pass)

1. Get  $Q, K, V \in \mathbb{R}^{N \times d}$  (in HBM) and the SRAM size  $M$  (about 100 KB).

2. **Determine block sizes:**

$$B_c = \left\lceil \frac{M}{4d} \right\rceil, \quad B_r = \min \left( \left\lceil \frac{M}{4d} \right\rceil, d \right).$$

For  $d = 64$ , 4 bytes per element and  $M = 100$  KB:  $B_c = \frac{100 \times 1024 \text{ B}}{4 \times 64 \times 4 \text{ B}} = 100$  and  $B_r = \min(100, 64) = 64$ .

3. **Initialise in HBM:**  $O = (0)_{N \times d}$ ,  $l = (0)_N \in \mathbb{R}^N$ ,  $m = (-\infty)_N \in \mathbb{R}^N$ .

4. **Divide**  $Q$  into  $T_r = \lceil N/B_r \rceil$  blocks  $Q_1, \dots, Q_{T_r}$  with  $Q_i \in \mathbb{R}^{B_r \times d}$ , and  $K, V$  into  $T_c = \lceil N/B_c \rceil$  blocks  $K_j, V_j \in \mathbb{R}^{B_c \times d}$ .

5. **Divide**  $O$  into  $T_r$  blocks  $O_i \in \mathbb{R}^{B_r \times d}$ , and  $l, m$  into  $T_r$  segments  $l_i, m_i \in \mathbb{R}^{B_r}$ .

#### Note: why $M/4d$ ?

SRAM must hold several tiles at once: a  $K_j$  and  $V_j$  tile ( $B_c \times d$  each) and a  $Q_i$  and  $O_i$  tile ( $B_r \times d$  each), plus the small  $B_r \times B_c$  score tile. Keeping roughly four  $B \times d$  tiles within  $M$  gives  $4Bd \leq M$ , i.e.  $B \approx M/(4d)$ . Capping  $B_r$  at  $d$  keeps the  $B_r \times B_c$  score tile  $S_{ij}$  within budget as well. (The paper measures  $M$  in elements; the slides divide bytes by 4 bytes per element, hence the extra factor of 4 in the worked number.)

**Algorithm 1** FlashAttention forward pass (outer loop over  $K/V$  blocks, inner loop over  $Q$  blocks)

**Require:**  $Q, K, V \in \mathbb{R}^{N \times d}$  in HBM; SRAM size  $M$

```

1:  $B_c \leftarrow \lceil M/4d \rceil$ ,  $B_r \leftarrow \min(\lceil M/4d \rceil, d)$ 
2: Initialise  $O \leftarrow 0_{N \times d}$ ,  $l \leftarrow 0_N$ ,  $m \leftarrow (-\infty)_N$  in HBM
3: Split  $Q$  into  $T_r$  blocks;  $K, V$  into  $T_c$  blocks;  $O, l, m$  into  $T_r$  blocks
4: for  $j = 1, \dots, T_c$  do ▷ iterate over blocks of  $K, V$ 
5:   Load  $K_j, V_j$  from HBM to SRAM
6:   for  $i = 1, \dots, T_r$  do ▷ iterate over blocks of  $Q$ 
7:     Load  $Q_i, O_i, l_i, m_i$  from HBM to SRAM
8:     On chip:  $S_{ij} = Q_i K_j^\top \in \mathbb{R}^{B_r \times B_c}$ 
9:     On chip:  $\tilde{m}_{ij} = \text{rowmax}(S_{ij})$ ,  $\tilde{P}_{ij} = \exp(S_{ij} - \tilde{m}_{ij})$ ,  $\tilde{l}_{ij} = \text{rowsum}(\tilde{P}_{ij})$ 
10:    On chip:  $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij})$ ,  $l_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} l_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{l}_{ij}$ 
11:    Write  $O_i \leftarrow \text{diag}(l_i^{\text{new}})^{-1} (\text{diag}(l_i) e^{m_i - m_i^{\text{new}}} O_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{P}_{ij} V_j)$  to HBM
12:    Write  $m_i \leftarrow m_i^{\text{new}}$ ,  $l_i \leftarrow l_i^{\text{new}}$  to HBM
13:   end for
14: end for
15: return  $O$ 
```

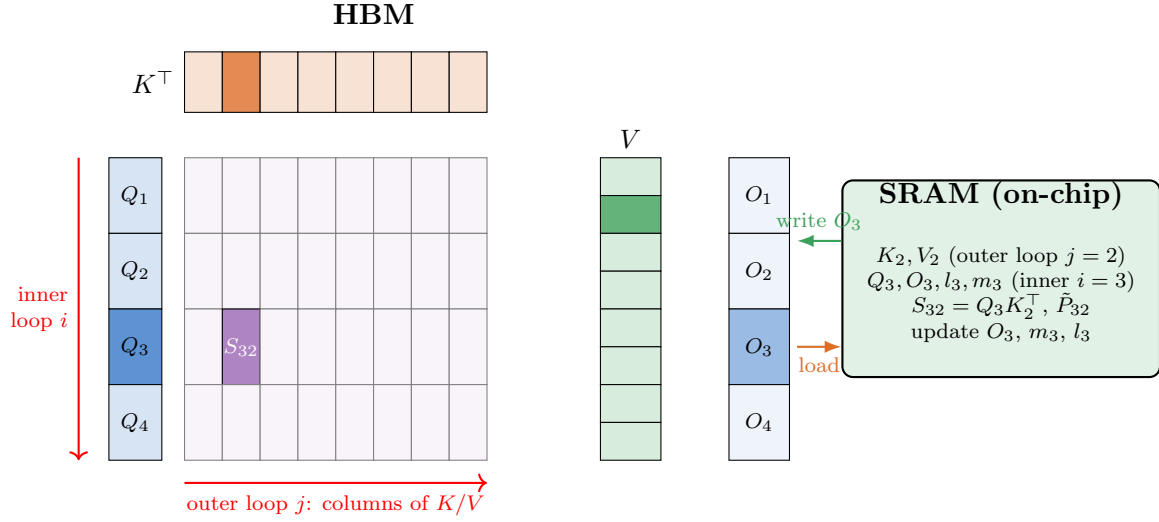


Figure 36: One step of FlashAttention ( $j = 2, i = 3$ ). Only tiles live in SRAM. The full  $N \times N$  matrix  $S$  is never stored anywhere; each tile is computed, used and discarded.

### 5.11 Walk-through: $N = 128, d = 16, B_c = 16, B_r = 32$

$$Q_i \in \mathbb{R}^{32 \times 16}, \quad K_j \in \mathbb{R}^{16 \times 16}, \quad V_j \in \mathbb{R}^{16 \times 16}, \quad O \in \mathbb{R}^{128 \times 16} \text{ (initialised to zeros),}$$

$$T_r = \lceil 128/32 \rceil = 4 \text{ query blocks}, \quad T_c = \lceil 128/16 \rceil = 8 \text{ key/value blocks.}$$

| $j$ (outer) | $i$ (inner) | tiles in SRAM        | what happens                                                                                        |
|-------------|-------------|----------------------|-----------------------------------------------------------------------------------------------------|
| 1           | 1           | $K_1, V_1; Q_1$      | $S_{11} = Q_1 K_1^\top$ ( $32 \times 16$ ), compute $m_1, l_1$ , write first partial $O_1$          |
| 1           | 2           | $K_1, V_1; Q_2$      | same for rows 33–64: partial $O_2$                                                                  |
| 1           | 3           | $K_1, V_1; Q_3$      | partial $O_3$                                                                                       |
| 1           | 4           | $K_1, V_1; Q_4$      | partial $O_4$ . Now every row has statistics from $K_1$ only                                        |
| 2           | 1           | $K_2, V_2; Q_1$      | $S_{12}$ : update $m_1, l_1$ ; rescale old $O_1$ ( $\beta_{11}$ ) and add new part ( $\beta_{12}$ ) |
| 2           | 2–4         | $K_2, V_2; Q_2..Q_4$ | same rescale-and-add for $O_2, O_3, O_4$                                                            |
| 3           | 1–4         | $K_3, V_3; \dots$    | iteratively update the statistics and keep refining each $O_i$                                      |
| $\vdots$    | $\vdots$    |                      |                                                                                                     |
| 8           | 4           | $K_8, V_8; Q_4$      | last update: $O$ is now the <b>final output of this head</b>                                        |

There are  $T_c \times T_r = 8 \times 4 = 32$  tile steps in total. Each  $O_i$  is refined 8 times, once per key/value block, and after the last one it equals exact attention.

|       | $K_1$ | $K_2$ | $K_3$ | $K_4$ | $K_5$ | $K_6$ | $K_7$ | $K_8$ |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| $Q_1$ | 1     | 5     | 9     | 13    | 17    | 21    | 25    | 29    |
| $Q_2$ | 2     | 6     | 10    | 14    | 18    | 22    | 26    | 30    |
| $Q_3$ | 3     | 7     | 11    | 15    | 19    | 23    | 27    | 31    |
| $Q_4$ | 4     | 8     | 12    | 16    | 20    | 24    | 28    | 32    |

numbers = processing order  
(column by column: outer  $j$ , inner  $i$ )

Figure 37: Order in which the  $4 \times 8$  tiles of  $S$  are visited in the walk-through.

## 5.12 HBM access complexity

- **Standard attention** requires  $\Theta(Nd + N^2)$  HBM accesses: read  $Q, K, V$  ( $Nd$ ), then write and re-read  $S$  and  $P$  ( $N^2$ ).
- **FlashAttention** requires  $\Theta\left(\frac{N^2 d^2}{M}\right)$  HBM accesses.

### Derivation: FlashAttention's $\mathcal{O}(N^2 d^2 / M)$

Each  $K_j, V_j$  is loaded once:  $\mathcal{O}(Nd)$  in total. For each of the  $T_c$  outer iterations, the inner loop streams *all* of  $Q$  and  $O$  through SRAM:  $\mathcal{O}(Nd)$  per outer iteration. With  $T_c = N/B_c = \mathcal{O}(Nd/M)$  (since  $B_c = \Theta(M/d)$ ):

$$\text{HBM accesses} = \mathcal{O}(Nd) + T_c \cdot \mathcal{O}(Nd) = \mathcal{O}\left(\frac{Nd}{M} \cdot Nd\right) = \mathcal{O}\left(\frac{N^2 d^2}{M}\right).$$

Since typically  $d^2 \ll M$  (e.g.  $d = 64$ :  $d^2 = 4096$ , while  $M \approx 100 \text{ KB} \approx 25,600 \text{ fp32 elements}$ ), this is several times less than  $N^2$ .

### Worked example: how many fewer accesses?

$N = 4096$ ,  $d = 64$ ,  $M = 25,600$  elements. Standard:  $Nd + N^2 \approx 0.26\text{M} + 16.8\text{M} \approx 17.0\text{M}$  element accesses (per read or write pass, ignoring constants). Flash:  $N^2 d^2 / M = 16.8\text{M} \times 4096 / 25600 \approx 2.7\text{M}$ . That is about  $6\times$  fewer HBM accesses, and more for larger SRAM or smaller  $d$ . The FLOPs are the same, but the wall-clock time drops because memory traffic was the bottleneck.

## 5.13 Recomputation (backward pass)

The backward pass needs  $S = QK^\top$  and  $P = \text{softmax}(S)$ . A fused kernel does *not* store these  $N \times N$  intermediates. How do we recover them?

**Recompute them**, block by block, in the backward pass from  $Q, K, V$  (still in HBM) and the saved per-row statistics  $m$  and  $l$  (and  $O$ ). Storing  $m, l$  costs only  $\mathcal{O}(N)$ .

### Intuition

Recomputation trades extra FLOPs for fewer memory accesses. Since attention is memory-bound, recomputing  $S_{ij}$  on-chip is *faster* than reading a stored  $S_{ij}$  back from HBM. The backward pass does more FLOPs but finishes sooner, and the  $\mathcal{O}(N^2)$  activation memory disappears.

## 5.14 Did we achieve a wall-clock speed-up?

Yes. **For the same compute budget, the context length can be increased from 1K to 4K** using FlashAttention: GPT-2 trained with FlashAttention at 4K context was still faster than a standard (Megatron) implementation at 1K. Memory grows only linearly in  $N$ , so long sequences fit on a GPU.

**Summary: FlashAttention in one table**

|              | Standard attention            | FlashAttention                                       |
|--------------|-------------------------------|------------------------------------------------------|
| FLOPs        | $\mathcal{O}(N^2d)$           | $\mathcal{O}(N^2d)$ (plus recompute in backward)     |
| Extra memory | $\mathcal{O}(N^2)$ ( $S, P$ ) | $\mathcal{O}(N)$ ( $m, l$ )                          |
| HBM accesses | $\Theta(Nd + N^2)$            | $\Theta(N^2d^2/M)$                                   |
| Exact?       | yes                           | yes                                                  |
| Key tricks   | —                             | tiling, online softmax, kernel fusion, recomputation |

**5.15 Another memory-bound case: inference**

**Question:** can you think of one more common scenario where the operation is memory-bound?

*Hint: not during training.*

**Answer: model inference with autoregressive generation.**

- There are many places where tiling and recomputation can reduce HBM accesses.
- But FlashAttention, which parallelises across queries and batch, cannot be used directly in decoding: we do not have whole  $Q, K, V$  matrices. They are built up gradually, one token per time step, and each step has a *single* query.
- So we need a different mechanism to reduce memory accesses. Methods for inference optimisation by reducing memory access (CPU/GPU) were proposed years before FlashAttention (see Module 5).
- Recently, a modified FlashAttention that **parallelises across keys/values and sequence length** was adapted for inference: **Flash-Decoding**.

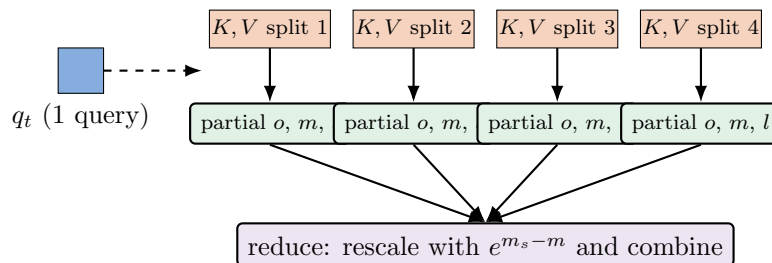


Figure 38: Flash-Decoding: with only one query per step, split the long KV sequence into chunks processed in parallel, then merge the partial results with the same online-softmax rescaling.

## 6 Module 5: Fast inference

### 6.1 Naive autoregressive inference

Take a closer look at a naive implementation of autoregressive (decoder) inference to see why the inference pipeline is worth optimising. Generate “I like coffee” from a [start] token:

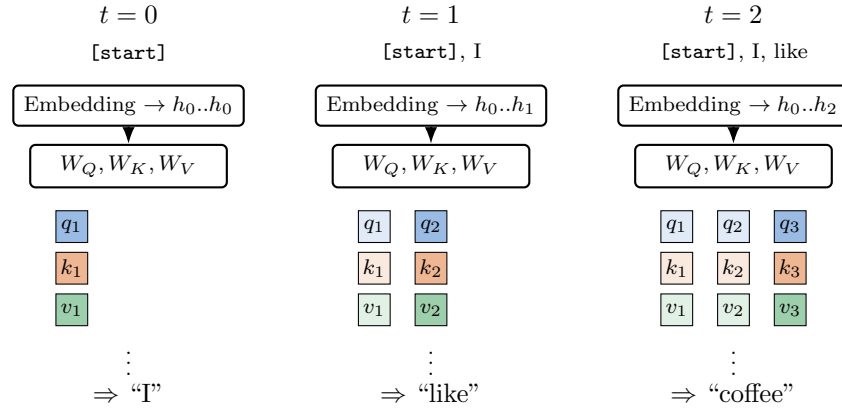


Figure 39: Naive decoding. At every step the model recomputes queries, keys and values for *all* previous tokens (pale), although only the newest token’s vectors (dark) are new.

- At each time step we recompute the keys and values for **all previous tokens** as well. This wastes compute and increases latency.
- Moreover, processing an LLM request can be about **10× more expensive** than a traditional keyword query. Can we do better?

#### Note: why only the newest query matters

In a causal decoder, the representation of token  $s$  at every layer depends only on tokens  $\leq s$ . When token  $t$  is appended, the keys and values of tokens  $1..t-1$  **do not change**. And to predict the next token we only need the output at position  $t$ , which needs only the *new* query  $q_t$ . Recomputing  $q_1..q_{t-1}$ ,  $k_1..k_{t-1}$ ,  $v_1..v_{t-1}$  is pure waste.

### 6.2 KV caching

1. At the first time step, start with a query, and compute and store its key–value pair  $(k_1, v_1)$  in a **cache**. The cache lives in GPU RAM, which is HBM.
2. At time step 2, compute only  $q_2, k_2, v_2$ ; **reload**  $k_1, v_1$  from HBM; attend with  $q_2$  over  $\{k_1, k_2\}$ ,  $\{v_1, v_2\}$ ; append  $(k_2, v_2)$  to the cache.
3. At time step 3, compute  $q_3, k_3, v_3$ , reload  $k_1, v_1, k_2, v_2$ , attend, and append  $(k_3, v_3)$ . And so on.

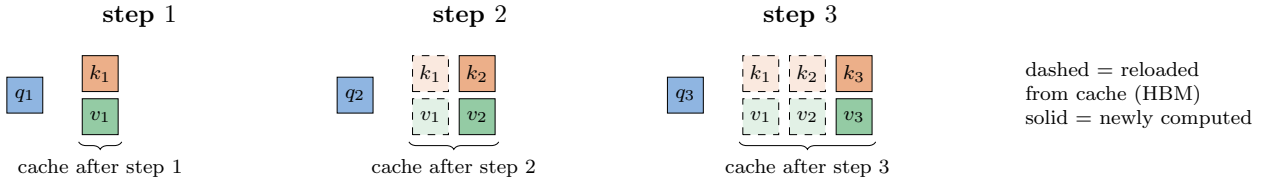


Figure 40: KV caching: only one new  $(k_t, v_t)$  pair per step is computed; older pairs are read from the cache.

### Key idea

KV caching **trades memory for compute**. The per-step projection work becomes constant, so the total projection compute for generating  $n$  tokens scales linearly,  $\mathcal{O}(n)$ , instead of quadratically.

### Derivation: compute saved by the KV cache (per layer)

Let the per-token projection cost be  $c_p = \mathcal{O}(d_{\text{model}}^2)$ .

- **Naive:** at step  $t$  we project all  $t$  tokens, so the total over  $n$  steps is  $\sum_{t=1}^n t c_p = \frac{n(n+1)}{2} c_p = \mathcal{O}(n^2 d_{\text{model}}^2)$ .
- **Cached:** one token per step,  $n c_p = \mathcal{O}(n d_{\text{model}}^2)$ .

The attention itself at step  $t$  is still  $q_t$  against  $t$  keys,  $\mathcal{O}(t d)$ , so  $\mathcal{O}(n^2 d)$  in total. The naive version is even worse, because it recomputes full  $t \times t$  attention at every step:  $\sum_t t^2 d = \mathcal{O}(n^3 d)$ .

Since the KV cache uses the same GPU RAM where the model weights are stored, **we cannot let it grow indefinitely**. How big does it get?

## 6.3 KV cache size

$$\text{KV cache per token (bytes)} = 2 \times \text{num\_layers} \times (\text{num\_heads} \times \text{dim\_head}) \times \text{precision\_in\_bytes}$$

The factor 2 is for keys *and* values.  $\text{num\_heads} \times \text{dim\_head} = d_{\text{model}}$  for standard multi-head attention.

### Worked example: GPT-3

num\_layers = 96, num\_heads = 96, dim\_head = 128, precision = 4 bytes (fp32).

per token =  $2 \times 96 \times (96 \times 128) \times 4 = 9,437,184 \text{ B} \approx 9.4 \text{ MB}$ ,

2K context =  $2048 \times 9.4 \text{ MB} \approx 19.3 \text{ GB}$ .

Generating a sequence of length 2K needs **19.3 GB for the KV cache alone**. We cannot even use a 40 GB A100, which must also hold the weights (175B parameters, i.e. 350 GB in fp16, spread over many GPUs), and the cache is per sequence, so it multiplies by the batch size.

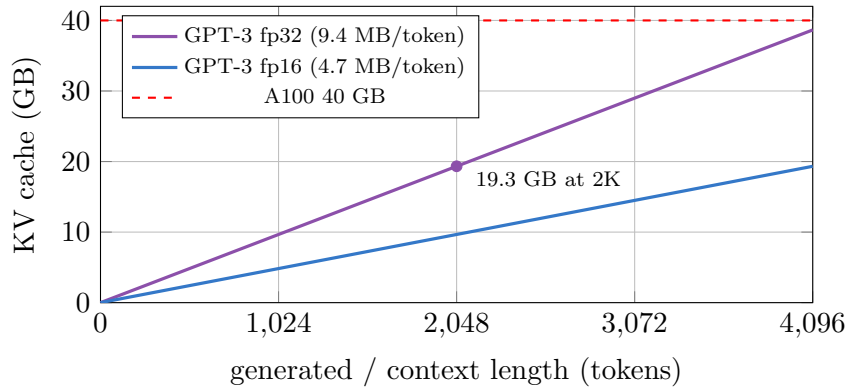


Figure 41: The KV cache grows linearly with the number of tokens, for every sequence in the batch.

Two families of fixes:

1. **Drop old entries:** discard the keys and values of tokens from the distant past, which behaves like local windowed attention (a sliding-window cache).
2. **Model-level modifications:** change the architecture so that fewer keys/values are stored. This gives MQA and GQA.

## 6.4 Multi-Query Attention (MQA)

Which of the three hyperparameters in the formula (`num_layers`, `num_heads`, `dim_head`) could we exploit to reduce memory? The answer is `num_heads`.

Set `num_heads = 1` in the *KV-cache formula*. This does **not** mean we use only one head. The factor 2 means we access two vectors (key and value) for each head, and we now want to access key–value pairs *once for all heads*. How? **Share the keys and values across all heads:** each head keeps its own query projection, but all heads use one shared  $K$  and one shared  $V$ :

$$\text{head}_i = \text{softmax}\left(\frac{Q_i K^\top}{\sqrt{d}}\right)V, \quad i = 1, \dots, n_h \quad (K, V \text{ shared}).$$

- This greatly reduces the cache memory, by a factor of  $n_h$  (e.g.  $96\times$  for GPT-3’s shape).
- However, **performance degrades significantly**.
- Converting an existing model requires re-training on at least about 5% of the original pre-training data. (The GQA paper calls this *uptraining*: mean-pool the existing K/V heads into the shared head, then continue training for about 5% of the original compute.)

## 6.5 Grouped-Query Attention (GQA)

- At one extreme, **Multi-Head Attention**: every head has its own query, key and value.
- At the other extreme, **Multi-Query Attention**: every head has its own query, but keys and values are shared across all heads.
- **Grouped-Query Attention** is the middle ground: the  $n_h$  query heads are divided into  $g$  groups, and **each group shares one key–value head**. Its quality is close to MHA and its speed close to MQA.

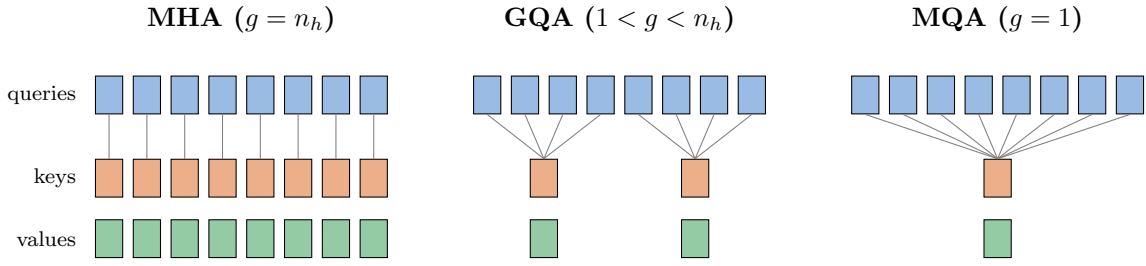


Figure 42: Eight query heads with 8, 2 and 1 key–value heads. The KV cache scales with the number of *key–value* heads.

#### Derivation: cache size under MHA / GQA / MQA

$$\text{KV cache per token} = 2 \times L \times g \times d_{\text{head}} \times \text{bytes}, \quad g = \begin{cases} n_h & \text{MHA} \\ \text{groups} & \text{GQA} \\ 1 & \text{MQA} \end{cases}$$

Reduction factor relative to MHA:  $n_h/g$ .

#### Worked example: a 70B-class model with GQA

$L = 80$ ,  $n_h = 64$  query heads,  $d_{\text{head}} = 128$ , fp16 (2 bytes). (These are the published LLaMA-2-70B dimensions, which uses GQA with  $g = 8$ .)

MHA ( $g = 64$ ) :  $2 \times 80 \times 64 \times 128 \times 2 = 2,621,440 \text{ B} \approx 2.6 \text{ MB/token}$ ,

GQA ( $g = 8$ ) :  $2 \times 80 \times 8 \times 128 \times 2 = 327,680 \text{ B} \approx 0.33 \text{ MB/token}$  ( $8 \times$  smaller),

MQA ( $g = 1$ ) :  $40,960 \text{ B} \approx 0.04 \text{ MB/token}$  ( $64 \times$  smaller).

For a 4K context, the per-sequence cache is about 10.7 GB (MHA), 1.3 GB (GQA) or 0.17 GB (MQA).

## 6.6 Paged Attention (vLLM)

- Typically, a **fixed, contiguous** chunk of memory is reserved for each request's KV cache, sized for the model's maximum context length.
- But we do not know in advance how long the generated sequence will be.
- If the model stops much earlier than the reserved length, the rest of the reservation is **wasted** and nobody else can use it.
- Solution:** store the KV cache in **blocks of non-contiguous memory**, allocated on demand. This is what Paged Attention does, borrowing the idea of *virtual memory paging* from operating systems.

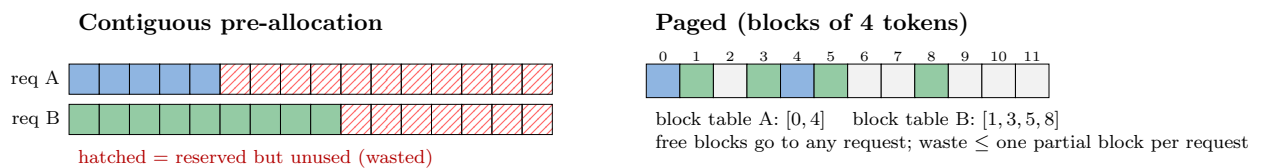


Figure 43: Left: fixed contiguous reservations waste memory. Right: paged KV cache. Each request owns a list of small physical blocks (its *block table*), which need not be adjacent.

**Intuition: the OS analogy**

Paged attention works like virtual memory. Tokens are like bytes, KV blocks are like pages, a request is like a process, and the block table is like the page table. The attention kernel follows the block table to gather the keys and values it needs. Wasted memory drops to at most one partially-filled block per sequence, so many more requests fit in a batch, and throughput rises. Blocks can even be *shared* between requests with a common prefix (e.g. the same system prompt, or several samples from one prompt) with copy-on-write.

**Summary: Module 5**

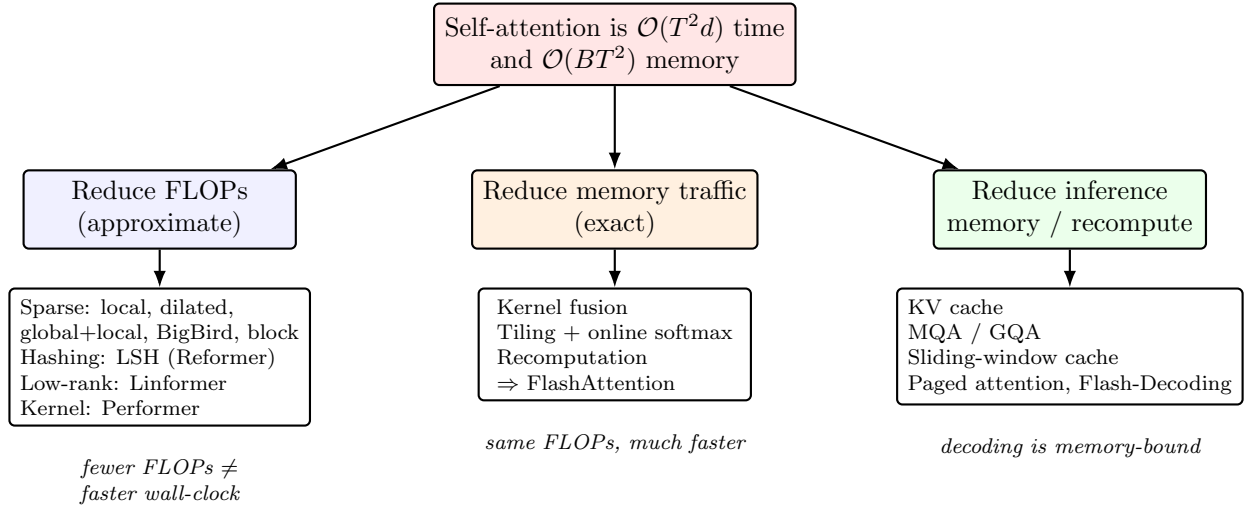
| Technique            | What it saves                                                                        | Cost / caveat                           |
|----------------------|--------------------------------------------------------------------------------------|-----------------------------------------|
| KV cache             | recomputing past $K, V$ ( $\mathcal{O}(n^2) \rightarrow \mathcal{O}(n)$ projections) | memory grows with length $\times$ batch |
| Sliding-window cache | bounds memory                                                                        | loses distant context                   |
| MQA                  | KV memory $\div n_h$                                                                 | quality drops; needs (up)training       |
| GQA                  | KV memory $\div (n_h/g)$                                                             | near-MHA quality; needs (up)training    |
| Paged attention      | fragmentation / over-reservation                                                     | needs a paged attention kernel          |
| Flash-Decoding       | parallelism over long KV at decode                                                   | —                                       |

**References (from the slides)**

- Generating Wikipedia by Summarizing Long Sequences
- Efficient Transformers: A Survey
- Long Range Arena: A Benchmark for Efficient Transformers
- Longformer: The Long-Document Transformer
- FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness
- Fast Transformer Decoding: One Write-Head is All You Need (MQA)
- GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints
- Efficient Memory Management for Large Language Model Serving with PagedAttention
- Reformer: The Efficient Transformer; Blog: A Deep Dive into Reformer
- Big Bird: Transformers for Longer Sequences

## 7 Consolidated summary and formula sheet

### 7.1 Big-picture concept map



### 7.2 Complexity cheat sheet

| Method                                | Time                                         | Score-matrix memory                 | Exact?        |
|---------------------------------------|----------------------------------------------|-------------------------------------|---------------|
| Full attention                        | $\mathcal{O}(T^2d)$                          | $\mathcal{O}(T^2)$                  | yes           |
| Strided local / dilated (window $c$ ) | $\mathcal{O}(cTd)$                           | $\mathcal{O}(cT)$                   | no            |
| Global + local ( $g$ global)          | $\mathcal{O}((c + 2g)Td)$                    | $\mathcal{O}((c + 2g)T)$            | no            |
| BigBird (+ $r$ random)                | $\mathcal{O}((c + 2g + r)Td)$                | $\mathcal{O}((c + 2g + r)T)$        | no            |
| Block attention ( $n$ blocks)         | $\mathcal{O}(T^2d/n)$                        | $\mathcal{O}(T^2/n)$                | no            |
| LSH / Reformer                        | $\mathcal{O}(T \log T)$                      | $\mathcal{O}(T \cdot \text{chunk})$ | no            |
| Linformer (rank $k$ )                 | $\mathcal{O}(kTd)$                           | $\mathcal{O}(kT)$                   | no            |
| Performer ( $r$ features)             | $\mathcal{O}(rTd)$                           | $\mathcal{O}(rT)$ (features)        | no (unbiased) |
| FlashAttention                        | $\mathcal{O}(T^2d)$ ; HBM $\Theta(T^2d^2/M)$ | $\mathcal{O}(T)$ extra              | yes           |

### 7.3 Formula sheet

| Quantity                          | Formula                                                                                                             |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------|
| One dot product                   | $d$ mults, $d - 1$ adds                                                                                             |
| $QK^\top$                         | $T^2d$ mults + $T^2(d - 1)$ adds $\approx 2T^2d$ FLOPs                                                              |
| $QKV$ memory (batch)              | $3BTd_{\text{model}}$                                                                                               |
| $Z$ or $A$ memory (batch)         | $Bn_hT^2$ each                                                                                                      |
| Receptive field, $L$ local layers | $L(c - 1) + 1$                                                                                                      |
| Block mask                        | $M_{ij} = 1 \iff \pi(\lfloor in/T \rfloor) = \lfloor jn/T \rfloor$ ; density $1/n$                                  |
| LSH hash                          | $h(x) = \arg \max(\text{concat}(xR, -xR)), R \in \mathbb{R}^{d_k \times b/2}$                                       |
| Linformer                         | $O = \text{softmax}(Q(EK)^\top / \sqrt{d})(FV), E, F \in \mathbb{R}^{k \times T}$                                   |
| Softmax as kernel                 | $\text{Atten} = D^{-1}A, A = \exp(QK^\top / \sqrt{d}), D = \text{diag}(A\mathbf{1}_T)$                              |
| Performer features                | $\phi(x) = \frac{1}{\sqrt{r}}e^{-\ x\ ^2/2}(e^{w_1^\top x}, \dots, e^{w_r^\top x}), w_i \sim \mathcal{N}(0, I_d)$   |
| Performer output                  | $\hat{D}^{-1}Q'(K'^\top V), \hat{D} = \text{diag}(Q'(K'^\top \mathbf{1}_T))$                                        |
| Safe softmax                      | $e^{x_i - m} / \sum_j e^{x_j - m}, m = \max x$                                                                      |
| Online softmax                    | $m = \max(m_1, m_2), l = e^{m_1 - m}l_1 + e^{m_2 - m}l_2$                                                           |
| Flash block sizes                 | $B_c = \lceil M/4d \rceil, B_r = \min(\lceil M/4d \rceil, d); T_r = \lceil N/B_r \rceil, T_c = \lceil N/B_c \rceil$ |
| HBM accesses                      | standard $\Theta(Nd + N^2)$ ; Flash $\Theta(N^2d^2/M)$                                                              |
| KV cache / token                  | $2 \times L \times (g \times d_{\text{head}}) \times \text{bytes}; g = n_h$ (MHA), 1 (MQA), groups (GQA)            |

### 7.4 The ten sentences to remember

1. Self-attention costs  $\mathcal{O}(T^2d)$  time because of the  $T \times T$  score matrix.
2. Attention memory scales linearly in batch size and quadratically in context length.
3. BERT's attention mostly looks at neighbours and is sparse, which justifies local attention.
4. Local, dilated, global and random patterns cut cost to  $\mathcal{O}(cTd)$ . Global tokens like [cls] restore full-sequence information flow.
5. Block attention costs  $\mathcal{O}(T^2d/n)$ . Permutations per head capture long-range links, but the identity permutation matters most, and perplexity rises with  $n$ .
6. LSH attention hashes shared queries/keys with random rotations, sorts by bucket and attends within buckets:  $\mathcal{O}(T \log T)$ .
7. Attention matrices are low-rank (long-tailed singular values), so Linformer projects  $K, V$  to length  $k$ :  $\mathcal{O}(kTd)$ .
8. Performer replaces  $\exp(q^\top k)$  with positive random features and uses associativity:  $\mathcal{O}(rTd)$ .
9. Fewer FLOPs do not mean faster: attention is memory-bound. FlashAttention (tiling + online softmax + fusion + recomputation) is exact and IO-aware.
10. Decoding is memory-bound: the KV cache avoids recompute, MQA/GQA shrink the cache, and paged attention stops the cache wasting memory.

## 8 Practice set

The questions come first and the solutions are collected afterwards, so you can attempt everything before checking. Unless stated otherwise, “FLOPs” counts multiplications and additions separately, and  $1 \text{ KB} = 1024 \text{ bytes}$  ( $1 \text{ MB}$ ,  $1 \text{ GB}$  written in decimal where noted).

### 8.1 Part A: Numerical problems

- N1. Counting FLOPs.** For one attention head with  $T = 2048$  and  $d = 64$ , compute (a) the exact number of multiplications and additions for  $QK^\top$ ; (b) the FLOPs for  $AV$ ; (c) the approximate FLOPs for the row-wise softmax (take 4 operations per element). Which term dominates?
- N2. Quadratic scaling.** The attention block of a model takes 10 ms at  $T = 1024$ . Assuming time is proportional to FLOPs, estimate the time at  $T = 4096$  and at  $T = 32,768$ . What happens if instead  $d$  is doubled at  $T = 1024$ ?
- N3. Training memory.** A model has  $d_{\text{model}} = 1024$ ,  $n_h = 16$ , and is trained with  $B = 4$ ,  $T = 4096$  in fp16. For one layer compute (a) the memory for  $Q, K, V$ ; (b) the memory for the attention matrix  $A$ ; (c) by what factor (b) grows if  $T$  is doubled, and by what factor if  $B$  is doubled.
- N4. Global + local cost.**  $T = 4096$ , window  $c = 512$ , and  $g = 8$  global tokens. Estimate the number of score entries computed and compare with full attention.
- N5. Receptive field.** (a) With strided local attention of window  $c = 5$  in all 12 layers, how many input positions can the top-layer token at the centre depend on? (b) With  $c = 3$ , how many layers are needed before one token can depend on an entire  $T = 4096$  sequence (ignoring global tokens)?
- N6. Block mask.**  $T = 15$ ,  $n = 3$ ,  $\pi = (2, 0, 1)$ . (a) Which key positions does token  $i = 7$  attend to? Token  $i = 13$ ? (b) How many non-zero entries does  $M$  have, and what fraction of  $T^2$  is that? (c) Is  $M_{3,12}$  equal to 0 or 1?
- N7. Block attention savings.** For  $T = 512$ ,  $d = 64$ , compare the multiplications in the score computation for full attention and for block attention with  $n = 4$ . How many distinct permutations are available for  $n = 4$ , and how many blocks does the  $QK^\top$  grid contain?
- N8. LSH hashing.** With  $b = 6$  buckets, a vector gives  $xR = (0.3, -0.7, 0.2)$ . (a) What is  $R$ 's shape if  $d_k = 64$ ? (b) Compute  $h(x)$ . (c) A second vector gives  $yR = (-0.1, 0.6, 0.4)$ . Do  $x$  and  $y$  collide in this round?
- N9. LSH collision rate.** Over 4 hashing rounds, three vectors have buckets  $a : (1, 3, 0, 2)$ ,  $b : (1, 3, 2, 2)$ ,  $c : (0, 1, 0, 3)$ . Which pair is most likely close? Give each pair's collision fraction.
- N10. Linformer.**  $T = 8192$ ,  $d = 64$ ,  $k = 256$ . (a) Give the shapes of  $E$ ,  $EK$ ,  $Q(EK)^\top$ ,  $FV$  and  $O$ . (b) By what factor does the score matrix shrink? (c) How many parameters do  $E$  and  $F$  add per layer if unshared?
- N11. Performer: a tiny example.**  $d = 2$ ,  $r = 2$ , random vectors  $w_1 = (1, 1)$ ,  $w_2 = (0, -1)$ . For  $x = (1, 0)$  and  $y = (0, 1)$  compute  $\phi(x)$ ,  $\phi(y)$  and the estimate  $\phi(x)^\top \phi(y)$ . Compare with the true value  $\exp(x^\top y)$  and explain the gap.
- N12. Performer complexity.**  $T = 16,384$ ,  $d = 64$ ,  $r = 256$ . What is the ratio of full-attention FLOPs to Performer FLOPs (up to constants)? What is the size of  $K^\top V$ ?
- N13. Online softmax.**  $x_A = [1, 3]$ ,  $x_B = [2, 5]$ . Compute  $m_1, l_1, m_2, l_2$ , the combined  $m, l$ ,  $\beta_1, \beta_2$ , and the final softmax  $P$ . Verify against a direct computation.

- N14. Online output update.** Using the scores in N13 with scalar values  $v_A = [10, 20]$  and  $v_B = [30, 40]$ , compute the partial output after block A and then the updated output after block B using the rescaling formula. Verify against  $\sum_i P_i v_i$ .
- N15. FlashAttention tiling.** SRAM  $M = 64$  KB,  $d = 128$ , fp16 (2 bytes/element). Using the slide's convention, compute  $B_c$  and  $B_r$ . For  $N = 8192$ , compute  $T_r$ ,  $T_c$  and the total number of tile steps.
- N16. HBM accesses.**  $N = 8192$ ,  $d = 128$ ,  $M = 102,400$  elements. Estimate the ratio of HBM accesses of standard attention to FlashAttention.
- N17. SRAM fit.**  $N = 2048$ ,  $d = 128$ , fp16. How much memory do  $Q$  and  $K$  together need? How much does the full score matrix  $S$  need? Compare with 100 KB of SRAM per core.
- N18. KV cache.** A 7B model has 32 layers, 32 heads and  $d_{\text{head}} = 128$ , served in fp16. (a) KV cache per token? (b) For a 4096-token context? (c) For a batch of 16 such sequences?
- N19. GQA.** Same model as N18 but converted to GQA with  $g = 4$  KV heads. New per-token cache size and reduction factor? What would MQA give?
- N20. Paged attention.** Maximum context 2048. Three requests finish at lengths 300, 1000 and 150 tokens. (a) With contiguous per-request reservation of 2048 slots, what fraction of reserved slots is wasted? (b) With paged blocks of 16 tokens?
- N21. Decoding compute.** Generating  $n = 1000$  tokens, how many token-level  $K/V$  projections per layer are computed (a) without and (b) with a KV cache?
- N22. GPT-3 KV cache.** Verify the slide's figures: per-token cache and cache for a 2K context (96 layers, 96 heads, 128 dims, fp32). What would fp16 give?

## 8.2 Part A: Solutions

- N1.** (a) Mults  $= T^2 d = 2048^2 \times 64 = 268,435,456$ . Adds  $= T^2(d - 1) = 264,241,152$ . Total  $\approx 5.33 \times 10^8$ . (b) AV:  $Td(2T - 1) = 2048 \times 64 \times 4095 = 536,739,840 \approx 5.37 \times 10^8$ . (c) Softmax  $\approx 4T^2 = 16,777,216 \approx 1.7 \times 10^7$ . The two matmuls dominate ( $\sim 2T^2 d$  each); softmax has about  $d/2 = 32 \times$  fewer FLOPs, though, as Module 4 shows, it can still dominate wall-clock time.
- N2.** Time  $\propto T^2$ .  $T = 4096$  is  $4 \times$  longer, so  $16 \times$ :  $\approx 160$  ms.  $T = 32,768$  is  $32 \times$  longer, so  $1024 \times$ :  $\approx 10.2$  s. Doubling  $d$  only doubles the cost:  $\approx 20$  ms.
- N3.**  $d = 1024/16 = 64$ . (a)  $3BTd_{\text{model}} = 3 \times 4 \times 4096 \times 1024 = 50.3\text{M numbers} \times 2 \text{ B} \approx 100.7 \text{ MB}$ . (b)  $Bn_h T^2 = 4 \times 16 \times 4096^2 = 1.07\text{G numbers} \times 2 \text{ B} \approx 2.15 \text{ GB}$  (another 2.15 GB for  $Z$  if stored). (c)  $T \rightarrow 2T$ :  $4 \times$  (quadratic).  $B \rightarrow 2B$ :  $2 \times$  (linear).
- N4.** Entries  $\approx (c + 2g)T = (512 + 16) \times 4096 = 2,162,688$ . Full:  $4096^2 = 16,777,216$ . Ratio  $\approx 12.9\%$ , about  $7.8 \times$  cheaper.
- N5.** (a)  $L(c - 1) + 1 = 12 \times 4 + 1 = 49$  positions. (b) Need  $L(c - 1) + 1 \geq T$ , i.e.  $2L + 1 \geq 4096$ , so  $L \geq 2047.5$ : **2048 layers**. This is why purely local models add global tokens or a global top layer.
- N6.** Block size  $= T/n = 5$ ; block of token  $i$  is  $\lfloor 3i/15 \rfloor = \lfloor i/5 \rfloor$ . (a)  $i = 7$ : block 1,  $\pi(1) = 0$ , so it attends to keys 0–4.  $i = 13$ : block 2,  $\pi(2) = 1$ , keys 5–9. (b)  $n(T/n)^2 = 3 \times 25 = 75$  non-zeros;  $75/225 = 1/3 = 1/n$ . (c)  $i = 3$ : block 0,  $\pi(0) = 2$ .  $j = 12$ : block 2. Equal, so  $M_{3,12} = 1$ .

- N7.** Full:  $T^2d = 512^2 \times 64 = 16,777,216$ . Block ( $n = 4$ ):  $T^2d/n = 4,194,304$ , a  $4\times$  saving. Permutations:  $4! = 24$ . Blocks in the grid:  $n^2 = 16$ .
- N8.** (a)  $R \in \mathbb{R}^{d_k \times b/2} = \mathbb{R}^{64 \times 3}$ . (b)  $\text{concat}(xR, -xR) = (0.3, -0.7, 0.2, -0.3, 0.7, -0.2)$ ; the max is 0.7 at index 4, so  $h(x) = 4$ . (c)  $\text{concat}(yR, -yR) = (-0.1, 0.6, 0.4, 0.1, -0.6, -0.4)$ ; the max is at index 1, so  $h(y) = 1$ . No collision. Indeed  $x$  is nearly aligned with  $-r_2$  and  $y$  with  $+r_2$ : they point in roughly opposite directions.
- N9.**  $a$  vs  $b$ : rounds 1,2,4 agree,  $3/4$ .  $a$  vs  $c$ : round 3 only,  $1/4$ .  $b$  vs  $c$ : none,  $0/4$ . So  $(a, b)$  is the closest pair.
- N10.** (a)  $E \in \mathbb{R}^{256 \times 8192}$ ,  $EK \in \mathbb{R}^{256 \times 64}$ ,  $Q(EK)^\top \in \mathbb{R}^{8192 \times 256}$ ,  $FV \in \mathbb{R}^{256 \times 64}$ ,  $O \in \mathbb{R}^{8192 \times 64}$ . (b)  $T^2/(Tk) = T/k = 32\times$  smaller ( $67.1\text{M} \rightarrow 2.1\text{M}$  entries). (c)  $2kT = 2 \times 256 \times 8192 = 4,194,304$  parameters per layer (per head if not shared).
- N11.**  $\|x\|^2 = \|y\|^2 = 1$ , so the prefactor is  $e^{-1/2}/\sqrt{2} = 0.4289$ .  $w_1^\top x = 1$ ,  $w_2^\top x = 0$ :  $\phi(x) = 0.4289(e^1, e^0) = (1.166, 0.429)$ .  $w_1^\top y = 1$ ,  $w_2^\top y = -1$ :  $\phi(y) = 0.4289(e^1, e^{-1}) = (1.166, 0.158)$ . Estimate:  $1.166 \times 1.166 + 0.429 \times 0.158 = 1.359 + 0.068 = 1.427$ . True:  $\exp(x^\top y) = e^0 = 1$ . The estimator is unbiased *in expectation*, but with only  $r = 2$  random features its variance is large. Increasing  $r$  (and orthogonalising  $W$ ) shrinks the error, with variance  $\propto 1/r$ .
- N12.** Full  $\sim T^2d$ , Performer  $\sim rTd$ , ratio  $T/r = 16384/256 = 64\times$ .  $K'^\top V \in \mathbb{R}^{r \times d} = \mathbb{R}^{256 \times 64}$ , independent of  $T$ .
- N13.**  $m_1 = 3$ ,  $f_1 = (e^{-2}, 1) = (0.1353, 1)$ ,  $l_1 = 1.1353$ .  $m_2 = 5$ ,  $f_2 = (e^{-3}, 1) = (0.0498, 1)$ ,  $l_2 = 1.0498$ .  $m = 5$ ,  $\beta_1 = e^{3-5} = 0.1353$ ,  $\beta_2 = 1$ .  $l = 0.1353 \times 1.1353 + 1.0498 = 1.2034$ .  $P = (0.1353 \cdot 0.1353, 0.1353, 0.0498, 1)/1.2034 = (0.0152, 0.1125, 0.0414, 0.8310)$ . Direct:  $e^x / \sum e^x$  with  $x = (1, 3, 2, 5)$  gives the same values (sum = 1).
- N14.** After block A:  $o^{(1)} = (0.1353 \times 10 + 1 \times 20)/1.1353 = 21.353/1.1353 = 18.81$ . Block B:  $\tilde{p}V_B = 0.0498 \times 30 + 1 \times 40 = 41.49$ . Update:  $o^{(2)} = (l_1 e^{m_1 - m} o^{(1)} + e^{m_2 - m} \cdot 41.49)/l = (1.1353 \times 0.1353 \times 18.81 + 41.49)/1.2034 = (2.890 + 41.49)/1.2034 = 36.88$ . Direct:  $0.0152 \cdot 10 + 0.1125 \cdot 20 + 0.0414 \cdot 30 + 0.8310 \cdot 40 = 36.88$ . ✓
- N15.**  $B_c = \lceil M/(4d \cdot \text{bytes}) \rceil = \lceil 65536/(4 \times 128 \times 2) \rceil = 64$ .  $B_r = \min(64, 128) = 64$ .  $T_r = 8192/64 = 128$ ,  $T_c = 8192/64 = 128$ , total tile steps =  $128 \times 128 = 16,384$ .
- N16.** Standard  $\approx N^2 = 67.1\text{M}$ . Flash =  $N^2 d^2 / M = 67.1\text{M} \times 16384 / 102400 = 10.7\text{M}$ . Ratio  $M/d^2 = 6.25\times$  fewer accesses.
- N17.**  $Q + K$ :  $2 \times 2048 \times 128 \times 2 \text{ B} = 1,048,576 \text{ B} = 1 \text{ MB}$ .  $S$ :  $2048^2 \times 2 \text{ B} = 8 \text{ MB}$ . Both are  $10\times$  to  $80\times$  larger than 100 KB, so tiling is unavoidable.
- N18.** (a)  $2 \times 32 \times (32 \times 128) \times 2 = 524,288 \text{ B} = 0.5 \text{ MiB}$  per token. (b)  $\times 4096 = 2.15 \text{ GB}$ . (c)  $\times 16 = 34.4 \text{ GB}$ , which together with the 14 GB of fp16 weights already exceeds a 40 GB GPU.
- N19.** GQA:  $2 \times 32 \times (4 \times 128) \times 2 = 65,536 \text{ B} = 64 \text{ KiB}$  per token, an  $8\times$  reduction ( $n_h/g = 32/4$ ). MQA ( $g = 1$ ):  $16,384 \text{ B} = 16 \text{ KiB}$  per token,  $32\times$ .
- N20.** (a) Reserved  $3 \times 2048 = 6144$ , used 1450, wasted 4694 slots: **76.4%**. (b) Blocks:  $\lceil 300/16 \rceil = 19$ ,  $\lceil 1000/16 \rceil = 63$ ,  $\lceil 150/16 \rceil = 10$ , so 92 blocks = 1472 slots. Waste = 22 slots, about 1.5%.
- N21.** (a)  $\sum_{t=1}^{1000} t = 500,500$  token projections. (b) 1000. Caching saves a factor of  $\approx 500$ .

**N22.**  $2 \times 96 \times (96 \times 128) \times 4 = 9,437,184 \text{ B} \approx 9.4 \text{ MB/token}$ ;  $\times 2048 \approx 19.3 \text{ GB}$ . fp16 halves it:  $\approx 4.7 \text{ MB/token}$  and  $\approx 9.7 \text{ GB}$ .

### 8.3 Part B: Subjective questions

- S1.** Why is self-attention quadratic in sequence length while the feed-forward layer is linear? At what sequence length does attention start to dominate a layer's FLOPs?
- S2.** Explain why reducing FLOPs by half does not halve training time. Use the terms *compute-bound*, *memory-bound* and *arithmetic intensity*.
- S3.** What empirical evidence justifies local/sparse attention? What do we lose by using it, and how do global tokens and random links compensate?
- S4.** Contrast strided local attention and dilated attention. Why do different heads often use different dilation rates?
- S5.** In block attention, what role does the permutation  $\pi$  play? Why is it useful to give different heads different permutations, and which permutation turned out most important?
- S6.** Explain why Reformer's LSH attention requires  $W_Q = W_K$ . What problem does this create, and how is it handled?
- S7.** Describe the complete LSH attention pipeline and explain why sorting by bucket is essential.
- S8.** What does it mean for the attention matrix to be low-rank? How did the Linformer authors test this, and how does Linformer exploit it? State two limitations.
- S9.** Explain how the Performer turns softmax attention into linear-time attention. Why must the random features be positive, and why orthogonalise  $W$ ?
- S10.** Why can't the softmax be computed tile by tile naively? Explain how online softmax solves this, and why the result is exact.
- S11.** List the four key ingredients of FlashAttention and the problem each one solves.
- S12.** FlashAttention does *more* FLOPs in the backward pass than standard attention yet runs faster. Explain.
- S13.** Why is autoregressive decoding memory-bound, and why can't FlashAttention be applied directly to it? What does Flash-Decoding change?
- S14.** Compare MHA, MQA and GQA in terms of KV-cache size, quality and training requirements.
- S15.** What problem does paged attention solve? Explain the operating-system analogy.

### 8.4 Part B: Model answers

- S1.** Attention computes an interaction score for every *pair* of tokens: the  $T \times T$  matrix  $QK^\top$  costs  $\mathcal{O}(T^2d)$ , and  $AV$  costs the same. The feed-forward layer processes each token independently ( $T$  separate  $d_{\text{model}} \rightarrow d_{\text{ff}} \rightarrow d_{\text{model}}$  maps), so it costs  $\mathcal{O}(Td_{\text{model}}d_{\text{ff}})$ , linear in  $T$ . Per layer the linear parts (projections + FFN) cost about  $12Td_{\text{model}}^2$  multiply-adds against  $2T^2d_{\text{model}}$  for attention, so attention dominates when  $T \gtrsim 6d_{\text{model}}$ . That is a few thousand tokens for typical model widths.
- S2.** Wall-clock time is set by whichever resource is saturated. A *compute-bound* kernel is limited by arithmetic throughput; a *memory-bound* kernel is limited by the bandwidth for moving data between HBM and the chip. *Arithmetic intensity* (FLOPs per byte) decides which regime applies. Standard attention materialises the  $T \times T$  matrices  $S$  and  $P$  and passes them through element-wise kernels (mask, softmax, dropout) that do about one FLOP per element moved, so it is memory-bound. Halving FLOPs leaves the memory traffic, and hence the time, largely

unchanged. Sparse methods can even add irregular memory access that GPUs handle poorly. Real speed-ups come from reducing memory traffic (FlashAttention).

- S3.** Studies of BERT’s attention maps (e.g. “What does BERT look at?”) found that many heads attend mostly to neighbouring tokens and that attention distributions are sparse, with most mass on a few keys. So computing all  $T^2$  entries is largely wasted. Local attention loses *direct* long-range interactions: information must hop through  $\sim T/c$  layers. Global tokens (e.g. [cls], chosen per task) attend to and are attended by everything, which gives every pair a 2-hop path and supports tasks needing whole-sequence summaries. Random links (BigBird) add graph shortcuts, making any two tokens a few hops apart, and help preserve the theoretical expressiveness of full attention.
- S4.** Both give each query  $c$  keys at cost  $\mathcal{O}(cTd)$ . Strided local attention uses a contiguous window  $[i - \lfloor c/2 \rfloor, i + \lfloor c/2 \rfloor]$ . Dilated attention spaces the window out  $(i, i \pm \delta, i \pm 2\delta, \dots)$  like a dilated convolution, covering a wider span at no extra cost but skipping positions in between. Mixing dilation rates across heads gives a multi-scale view: some heads capture fine local syntax, others coarser long-range context, and the union covers a wide receptive field without gaps.
- S5.** The permutation decides which key/value block each query block attends to: block  $k$  of queries attends to block  $\pi(k)$  of keys. The identity permutation gives purely local (block-diagonal) attention; other permutations connect distant blocks. With a different  $\pi$  per head, the union over heads covers many (query block, key block) pairs, so long-range dependencies are captured while each head costs only  $\mathcal{O}(T^2d/n)$ . Empirically the identity permutation was most important, consistent with the dominance of local context.
- S6.** LSH attention works only if a query and its best-matching keys land in the same bucket, which requires them to live in the same vector space and be hashed together. With independent  $W_Q, W_K$ , a query and a highly relevant key could point in quite different directions, hash to different buckets and never be compared. Sharing  $W_Q = W_K$  solves this. The side effect: each token’s dot product with itself is maximal, so it would attend mostly to itself. Reformer forbids attending to oneself unless no other target is available, and normalises keys. Empirically shared-QK costs little accuracy.
- S7.** (1) Compute random-rotation hashes for all tokens in parallel ( $QR$ , then  $\text{argmax}$  over  $\text{concat}(QR, -QR)$ ), optionally with  $n_R$  rounds. (2) Sort tokens by bucket and, within a bucket, by position. (3) Split the sorted sequence into fixed-size chunks. (4) Each query attends to keys in its own bucket within its chunk and the previous chunk, keeping causal masking where needed. Sorting is essential because same-bucket tokens are scattered through the original order. After sorting they are contiguous, so attention becomes a dense block-diagonal computation that runs efficiently on hardware, instead of a scattered gather. Overall cost is  $\mathcal{O}(T \log T)$ .
- S8.** A matrix of rank  $k$  can be written as a product of  $T \times k$  and  $k \times T$  factors, so its rows lie in a  $k$ -dimensional subspace. The Linformer authors ran SVD on RoBERTa attention matrices ( $512 \times 512$ ) across layers and heads, averaged over 10K sentences, and found a long-tailed singular value spectrum: a small number of singular values capture most of the energy, especially in higher layers. Linformer learns projections  $E, F \in \mathbb{R}^{k \times T}$  that compress the length dimension of  $K$  and  $V$  to  $k$ , giving a  $T \times k$  score matrix and  $\mathcal{O}(kTd)$  cost. Limitations: (i)  $E, F$  tie the model to a fixed sequence length; (ii) compression across positions breaks causal masking, so it is awkward for autoregressive decoders; (iii) it is an approximation.
- S9.** Softmax attention is  $D^{-1}A$  with  $A_{ij} = \exp(q_i^\top k_j / \sqrt{d})$ , a positive kernel. The Performer writes  $\exp(q^\top k) = \mathbb{E}[\phi(q)^\top \phi(k)]$  using random features  $\phi(x) = \frac{1}{\sqrt{r}} e^{-\|x\|^2/2} (e^{w_i^\top x})_i$ . Then  $A \approx Q'K'^\top$ , and associativity lets us compute  $Q'(K'^\top V)$  in  $\mathcal{O}(rTd)$  without forming the  $T \times T$  matrix; the

normaliser is  $Q'(K'^\top \mathbf{1})$ . The features must be positive because attention weights and their row sums must be positive: trigonometric features can give negative or near-zero denominators and high variance exactly where most attention entries are small, which destabilises training. Orthogonalising  $W$  (Gram–Schmidt,  $r \leq d$  per block) makes the random directions non-redundant and provably reduces the estimator’s variance.

- S10.** Softmax normalises by the sum over an entire row, and the max used for numerical safety also depends on the whole row. A tile only sees part of the row. Online softmax keeps, per row, the running max  $m$  and running sum  $l$  of exponentials relative to  $m$ . When a new tile arrives with local statistics  $(\tilde{m}, \tilde{l})$ , set  $m_{\text{new}} = \max(m, \tilde{m})$  and  $l_{\text{new}} = e^{m-m_{\text{new}}}l + e^{\tilde{m}-m_{\text{new}}}\tilde{l}$ , and rescale the partial output the same way. The rescaling factors exactly convert every stored exponential from the old reference max to the new one, because  $e^{x-m}e^{m-m'} = e^{x-m'}$ . After the last tile, the result is identical to the ordinary softmax up to floating-point rounding.
- S11.** (1) *Tiling*: split  $Q, K, V$  into blocks that fit in SRAM, because the full matrices do not fit. (2) *Online softmax*: running  $(m, l)$  statistics let a row’s softmax be assembled across tiles exactly, which solves the softmax-needs-the-whole-row problem. (3) *Kernel fusion*: matmul, mask, softmax, dropout and matmul run in one kernel, so intermediates never touch HBM; this removes the memory-bound chain of element-wise kernels. (4) *Recomputation*: instead of storing  $S$  and  $P$  ( $\mathcal{O}(N^2)$ ) for the backward pass, store only  $O$  and  $(m, l)$  and recompute tiles of  $S, P$  on-chip; this solves the  $\mathcal{O}(N^2)$  activation memory and the HBM reads it would require.
- S12.** Attention is memory-bound, so runtime is driven by HBM traffic, not FLOPs. Reading a stored  $N \times N$  matrix from HBM is slower than recomputing its tiles on-chip from  $Q, K$  (which are read anyway) and the  $\mathcal{O}(N)$  statistics. The extra FLOPs are nearly free because compute units were idle waiting for memory. FlashAttention therefore wins on time and on memory.
- S13.** At each decoding step there is a single new query. The step must read all model weights and the entire KV cache from HBM to do a small amount of arithmetic (a matrix–vector product), which is very low arithmetic intensity. FlashAttention parallelises over queries (and batch/heads). With one query per step there is almost nothing to parallelise across, and the  $Q, K, V$  matrices are not available up front but grow token by token. Flash-Decoding instead parallelises over the *key/value sequence length*: it splits the long KV cache into chunks processed in parallel, then combines the partial results with the online-softmax rescaling.
- S14.** MHA:  $n_h$  key/value heads, cache  $2Ln_h d_{\text{head}}$  per token, best quality. MQA: 1 shared K/V head, cache  $n_h \times$  smaller, fastest decoding, but noticeable quality loss and it needs (re)training; converting an MHA checkpoint needs uptraining on about 5% of the pre-training compute. GQA:  $g$  groups, each sharing one K/V head, cache  $n_h/g \times$  smaller, quality close to MHA with most of MQA’s speed; it can also be obtained by uptraining an MHA checkpoint (mean-pooling heads within each group).
- S15.** Serving systems used to reserve one contiguous KV-cache region per request, sized for the maximum length, since the output length is unknown. Short outputs leave most of that region unused, and fragmentation prevents reuse, which limits batch size and throughput. Paged attention stores the KV cache in small fixed-size blocks allocated on demand anywhere in memory, and a per-request *block table* maps logical token positions to physical blocks. The analogy: tokens are bytes, blocks are pages, requests are processes, the block table is the page table. Waste drops to at most one partial block per request, and blocks can be shared across requests with a common prefix (copy-on-write).

## 8.5 Part C: Multiple-choice questions

- Q1.** The time complexity of computing full self-attention for one head is (a)  $\mathcal{O}(Td)$  (b)  $\mathcal{O}(Td^2)$  (c)  $\mathcal{O}(T^2d)$  (d)  $\mathcal{O}(T \log T)$
- Q2.** Computing one row of  $QK^\top$  requires (a)  $d$  mults,  $d - 1$  adds (b)  $Td$  mults,  $T(d - 1)$  adds (c)  $T^2$  mults (d)  $T$  mults,  $T - 1$  adds
- Q3.** The row-wise softmax over a  $T \times T$  score matrix costs (a)  $\mathcal{O}(T)$  (b)  $\mathcal{O}(T^2)$  (c)  $\mathcal{O}(T^2d)$  (d)  $\mathcal{O}(Td)$
- Q4.** Memory for the attention matrices of a batch scales as (a) linear in  $B$ , linear in  $T$  (b) quadratic in  $B$ , linear in  $T$  (c) linear in  $B$ , quadratic in  $T$  (d) independent of  $B$
- Q5.** Memory for  $Q, K, V$  across all heads for a single sample equals (a)  $3Td_{\text{model}}$  (b)  $3T^2$  (c)  $n_h T^2$  (d)  $3d_{\text{model}}^2$
- Q6.** A 10 s audio clip at 8 kHz has sequence length (a) 8000 (b) 30,000 (c) 80,000 (d) 800,000
- Q7.** In strided local attention with window  $c = 5$ , each token attends to (a) 5 tokens on each side (b) 2 tokens left, 2 right, and itself (c) every 5th token (d) 5 random tokens
- Q8.** Dilated attention compared with strided local attention with the same  $c$ : (a) is more expensive (b) has the same cost but a wider span (c) is cheaper (d) is exact
- Q9.** In global + local attention, the global tokens are (a) learned automatically by the model (b) chosen randomly each step (c) set manually based on the task (d) always the last token
- Q10.** BigBird’s attention pattern combines (a) local + LSH (b) global + random + local (c) low-rank + kernel (d) block + dilated
- Q11.** Block attention with  $n$  blocks has time complexity (a)  $\mathcal{O}(T^2d)$  (b)  $\mathcal{O}(T^2d/n)$  (c)  $\mathcal{O}(Td/n)$  (d)  $\mathcal{O}(nTd)$
- Q12.** For  $T = 32$ ,  $n = 16$  block attention, the fraction of non-zero mask entries is (a) 50% (b) 25% (c) 6.25% (d) 1.56%
- Q13.** With  $T = 15$ ,  $n = 3$ ,  $\pi = (0, 2, 1)$ , token 6 attends to keys (a) 0–4 (b) 5–9 (c) 10–14 (d) all
- Q14.** As the number of blocks  $n$  increases in block attention, (a) training time and perplexity both decrease (b) training time decreases, perplexity increases (c) both increase (d) nothing changes
- Q15.** LSH attention in the Reformer requires (a)  $W_Q = W_V$  (b)  $W_Q = W_K$  (c)  $W_K = W_V$  (d) no weight sharing
- Q16.** The Reformer hash function is (a)  $\text{sign}(x)$  (b)  $\arg \max(\text{concat}(xR, -xR))$  (c)  $x \bmod b$  (d) MinHash over shingles
- Q17.** With  $b$  buckets and  $d_k$ -dimensional vectors,  $R$  has shape (a)  $d_k \times b$  (b)  $b \times d_k$  (c)  $d_k \times b/2$  (d)  $b/2 \times b/2$
- Q18.** The complexity of LSH attention is (a)  $\mathcal{O}(T^2)$  (b)  $\mathcal{O}(T \log T)$  (c)  $\mathcal{O}(T)$  (d)  $\mathcal{O}(\log T)$
- Q19.** For a matrix to be low-rank, its singular values should follow (a) a uniform distribution (b) a long-tailed distribution (c) a Gaussian distribution (d) all equal values
- Q20.** In Linformer,  $E$  and  $F$  have shape (a)  $d \times k$  (b)  $k \times d$  (c)  $k \times T$  (d)  $T \times T$

- Q21.** Linformer's attention matrix  $\bar{A}$  has shape (a)  $T \times T$  (b)  $T \times k$  (c)  $k \times k$  (d)  $k \times d$
- Q22.** In the Performer, softmax attention is written as  $D^{-1}A$  where  $D$  is (a)  $\text{diag}(A\mathbf{1}_T)$  (b)  $A^\top A$  (c)  $\sqrt{d}I$  (d) the identity
- Q23.** Performer's feature map uses  $h(x) =$  (a)  $\exp(\|x\|^2)$  (b)  $\exp(-\|x\|^2/2)$  (c)  $\|x\|$  (d) 1
- Q24.** Orthogonalising the random matrix  $W$  in the Performer (a) makes attention exact (b) reduces the estimator's variance and needs  $r \leq d$  (c) makes  $r$  independent of  $T$  (d) removes the softmax
- Q25.** The main reason approximate attention methods often fail to speed up wall-clock training is that attention is (a) compute-bound (b) memory-bound (c) network-bound (d) disk-bound
- Q26.** Kernel fusion helps because (a) it reduces FLOPs (b) it avoids writing intermediates to main memory (c) it increases precision (d) it enables sparsity
- Q27.** In the naive attention implementation, which operations take longer than the matmul? (a) none (b) only softmax (c) dropout, softmax, masking (d) only the output projection
- Q28.** GPU SRAM compared with HBM is (a) larger and slower (b) smaller and faster (c) larger and faster (d) the same memory
- Q29.** Safe softmax subtracts the maximum because (a) it changes the output distribution (b) it prevents overflow; exp is accurate for non-positive inputs (c) it is faster (d) it makes the softmax sparse
- Q30.** In online softmax, when combining blocks with maxima  $m_1 < m_2$ , the first block is rescaled by (a)  $e^{m_2-m_1}$  (b)  $e^{m_1-m_2}$  (c)  $m_1/m_2$  (d) 1
- Q31.** FlashAttention's block sizes (slide convention) are (a)  $B_c = \lceil M/4d \rceil$ ,  $B_r = \min(\lceil M/4d \rceil, d)$  (b)  $B_c = B_r = \sqrt{M}$  (c)  $B_c = d$ ,  $B_r = M$  (d)  $B_c = N/M$
- Q32.** The HBM access complexity of FlashAttention is (a)  $\Theta(Nd + N^2)$  (b)  $\Theta(N^2d^2/M)$  (c)  $\Theta(N)$  (d)  $\Theta(Nd/M)$
- Q33.** FlashAttention recovers  $S$  and  $P$  for the backward pass by (a) storing them in CPU memory (b) recomputing them from  $Q, K, V$  and the stored statistics  $m, l$  (c) approximating them (d) not needing them
- Q34.** Is FlashAttention an approximation of attention? (a) Yes, low-rank (b) Yes, sparse (c) No, it is exact (d) Only in fp16
- Q35.** According to the slides, for the same compute budget FlashAttention lets context grow from (a) 512 to 1K (b) 1K to 4K (c) 4K to 64K (d) 2K to 1M
- Q36.** The KV cache per token formula is (a)  $L \times n_h \times d_{\text{head}}$  (b)  $2 \times L \times n_h \times d_{\text{head}} \times \text{bytes}$  (c)  $2 \times T \times d_{\text{model}}$  (d)  $L \times T^2$
- Q37.** GPT-3's KV cache for a 2K context in fp32 is about (a) 9.4 MB (b) 1.9 GB (c) 19.3 GB (d) 193 GB
- Q38.** Multi-Query Attention shares (a) queries across heads (b) keys and values across heads (c) weights across layers (d) the output projection only
- Q39.** Grouped-Query Attention with  $g = n_h$  is equivalent to (a) MQA (b) MHA (c) Linformer (d) no attention

- Q40.** Paged attention primarily addresses (a) quadratic compute (b) wasted and fragmented KV-cache memory (c) numerical overflow (d) positional encoding length
- Q41.** Flash-Decoding parallelises across (a) queries (b) batch only (c) keys/values along the sequence length (d) layers
- Q42.** The number of permutations of  $n = 4$  blocks is (a) 4 (b) 16 (c) 24 (d) 256

## 8.6 Part C: Answer key with explanations

- Q1.** (c)  $QK^\top$  and  $AV$  each cost  $\mathcal{O}(T^2d)$ .
- Q2.** (b)  $T$  dot products, each  $d$  mults and  $d - 1$  adds. (a) is a single entry.
- Q3.** (b)  $\mathcal{O}(T)$  per row times  $T$  rows. Only the matmults carry the factor  $d$ .
- Q4.** (c)  $Bn_hT^2$ .
- Q5.** (a)  $3n_hTd = 3Td_{\text{model}}$  since  $n_hd = d_{\text{model}}$ .
- Q6.** (c)  $10 \times 8000 = 80,000$  samples.
- Q7.** (b)  $\lfloor c/2 \rfloor = 2$  on each side plus the token itself: 5 keys.
- Q8.** (b) Same number of keys, spaced out, so the cost is unchanged.
- Q9.** (c) Chosen per task, e.g. [cls] for classification or question tokens for QA.
- Q10.** (b) Global + random + local.
- Q11.** (b)  $n$  blocks of cost  $(T/n)^2d$ .
- Q12.** (c) Density is  $1/n = 1/16 = 6.25\%$  (about 93.75% sparse).
- Q13.** (c) Token 6 is in block  $\lfloor 6 \cdot 3/15 \rfloor = 1$  and  $\pi(1) = 2$ , so keys 10–14.
- Q14.** (b) Sparser is faster but approximates full attention less well, so perplexity rises.
- Q15.** (b) Queries and keys must share a space to be hashed into the same buckets.
- Q16.** (b) The random-rotation hash. (d) is used for document deduplication, not attention.
- Q17.** (c)  $xR$  has  $b/2$  entries; with  $-xR$  there are  $b$  candidates.
- Q18.** (b) Dominated by sorting and the per-query approximate nearest-neighbour search.
- Q19.** (b) A few large singular values followed by a long tail of tiny ones.
- Q20.** (c) They compress the length dimension  $T$  to  $k$ .
- Q21.** (b)  $Q(EK)^\top$  is  $(T \times d)(d \times k)$ .
- Q22.** (a) Row sums of  $A$ , so  $D^{-1}$  normalises each row.
- Q23.** (b) This choice makes  $\mathbb{E}[\phi(q)^\top \phi(k)] = \exp(q^\top k)$  exactly.
- Q24.** (b) Orthogonal random features lower variance; at most  $d$  orthogonal vectors exist in  $\mathbb{R}^d$ .
- Q25.** (b) Time is dominated by HBM traffic, not arithmetic.
- Q26.** (b) FLOPs are unchanged; memory round-trips disappear.
- Q27.** (c) Element-wise operations are memory-bound.

- Q28.** (b) About 20 MB at 19 TB/s versus 40–80 GB at 1.5–2 TB/s.
- Q29.** (b) The output is mathematically identical; it avoids overflow.
- Q30.** (b) Convert from reference  $m_1$  to the new max  $m_2$ : multiply by  $e^{m_1-m_2} \leq 1$ .
- Q31.** (a) Four tiles of size  $B \times d$  must fit in  $M$ ;  $B_r$  is capped at  $d$ .
- Q32.** (b)  $T_c = \mathcal{O}(Nd/M)$  passes over  $Q, O$  of size  $Nd$ .
- Q33.** (b) Recomputation, which trades FLOPs for memory accesses.
- Q34.** (c) It computes exactly the same result as standard attention.
- Q35.** (b) 1K to 4K.
- Q36.** (b) The factor 2 is for keys and values.
- Q37.** (c)  $9.4 \text{ MB/token} \times 2048 \approx 19.3 \text{ GB}$ .
- Q38.** (b) Each head keeps its own query, but one K and one V are shared.
- Q39.** (b) One K/V head per query head is exactly MHA.
- Q40.** (b) Non-contiguous on-demand blocks remove over-reservation waste.
- Q41.** (c) With one query per step, it splits the KV sequence into parallel chunks.
- Q42.** (c)  $4! = 24$ . (16 is the number of blocks,  $n^2$ .)